



University of St.Gallen

Institute of Computer Science

# Behavioral Analysis and – Modeling

Process Mining

Session 4



From insight to impact.

# Outline

## Session 1: Introduction

- *Event Logs, Event Log Inspection, First Exploration*

## Session 2: Exploratory Log Analysis

- *Initial Analysis, Visualizations, Log Enrichments*

## Session 3: Process KPIs

- *Process Outcomes, Dashboards*

## Session 4: Behavioral Analysis and –Modeling

- *Variants, Process Maps, BPMN, Petri Nets*

## First part of Assignment 1 due (March 13)

## Session 5: Conformance Checking

- *Business Rules and Normative Process Models*

## Session 6: Performance Mining

- *Load and Demand, Bottlenecks*

## Session 7: Deviance Diagnostics

- *Explanations and Rule Mining*

## Assignment 1 due

## Assignment 1 Readout

# Why do we need to discover a process?

## **Prescriptive model does not exist:**

- low process **maturity**
- prescriptive model is **implicit** in the minds of different process stakeholders, where each stakeholder
  - sees only part of the process
  - has different expectations based on different goals

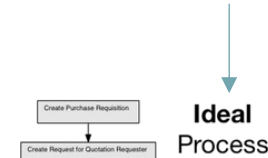
## **Prescriptive model does not specify everything:**

- it allows for **flexibility** (easy adaptation to context change)
- there are too many **exceptional behaviors** to specify

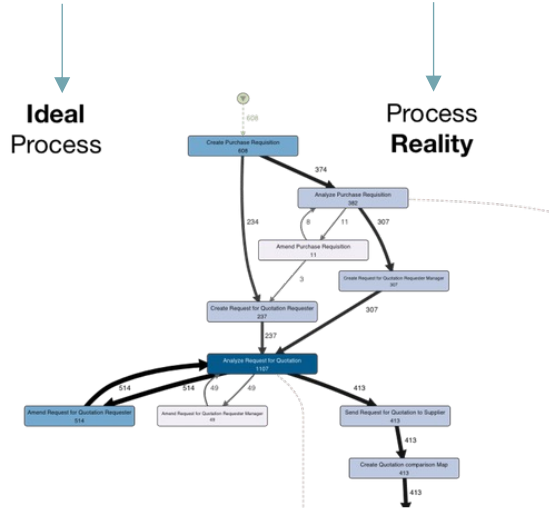
## **Prescriptive model is not strictly followed:**

- humans: misunderstanding or misaligned with prescription
- machines: incorrect implementation of the prescription

prescriptive model



descriptive model



Source: Fluxicon

# Desire Path



Source: [Wikipedia](#)

# Outline

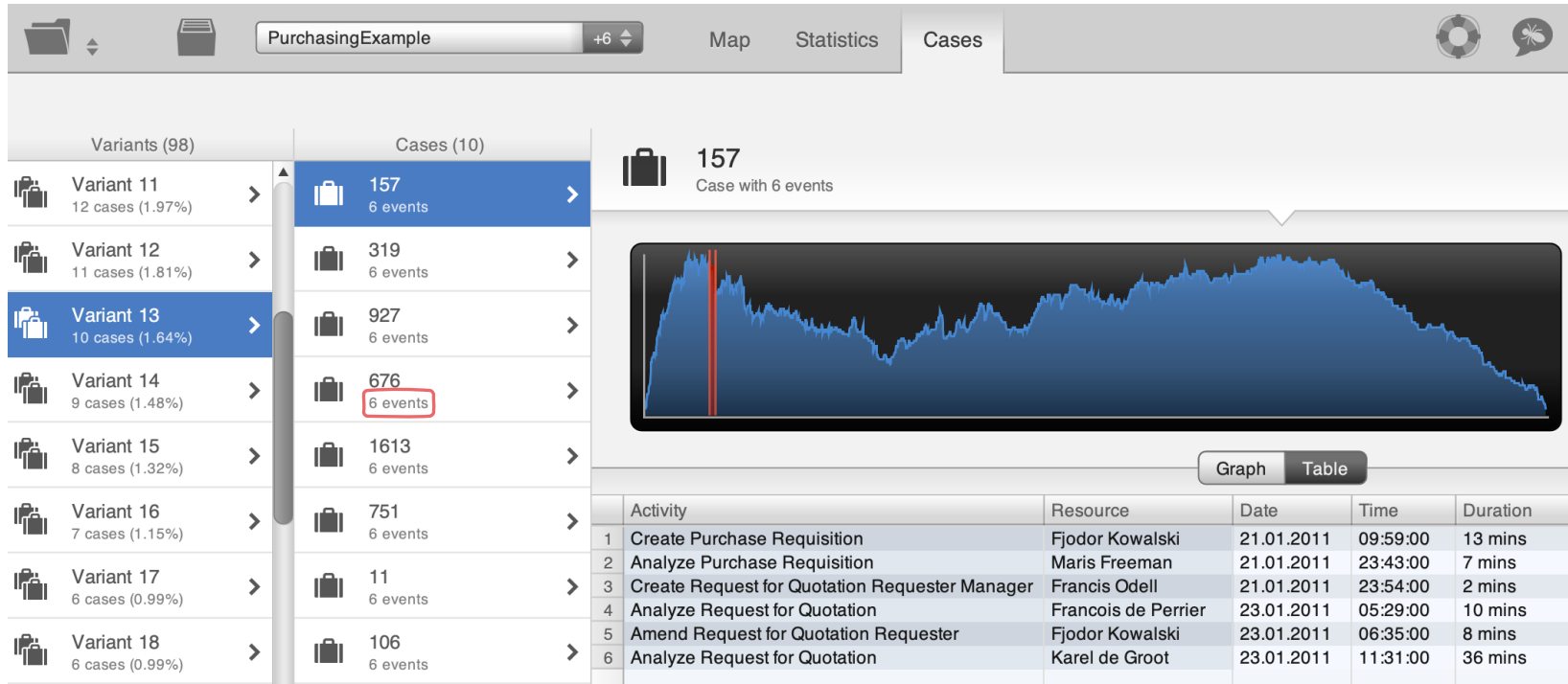
1. Sequential Variants
2. Process Maps
3. Process Modeling
  - BPMN, Petri Nets
4. Process Induction

# Sequential Variants

*Understand the process behaviour as a set of sequences of activities.*

# Sequential Variant

= the equivalence class of cases that have the same sequence of activities



partitioning of the log  
into variants

partitioning of a  
variant into cases

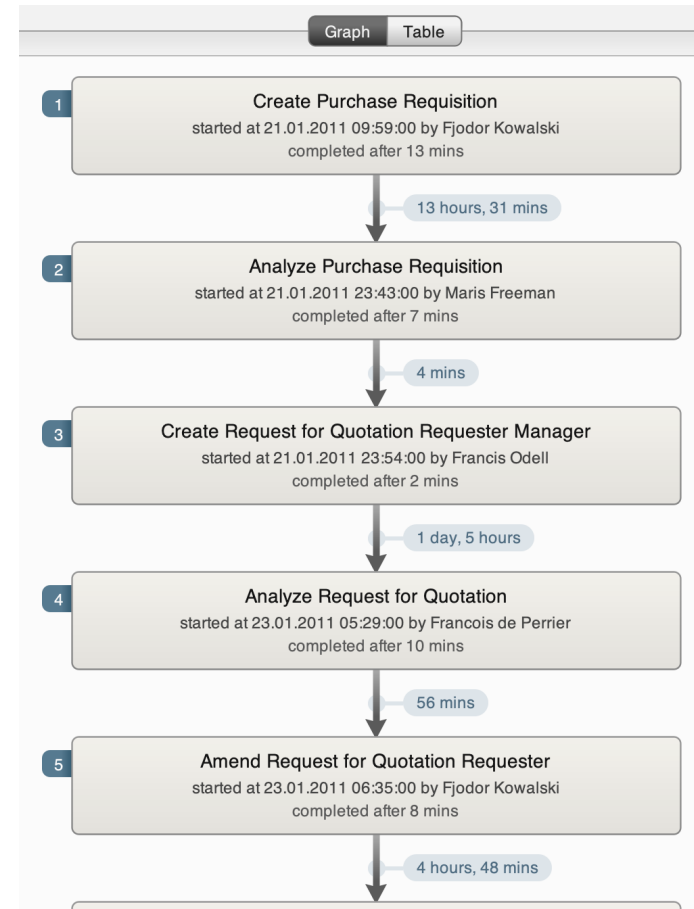
same sequence of activities for  
each case of the same variant

partitioning of a case into events

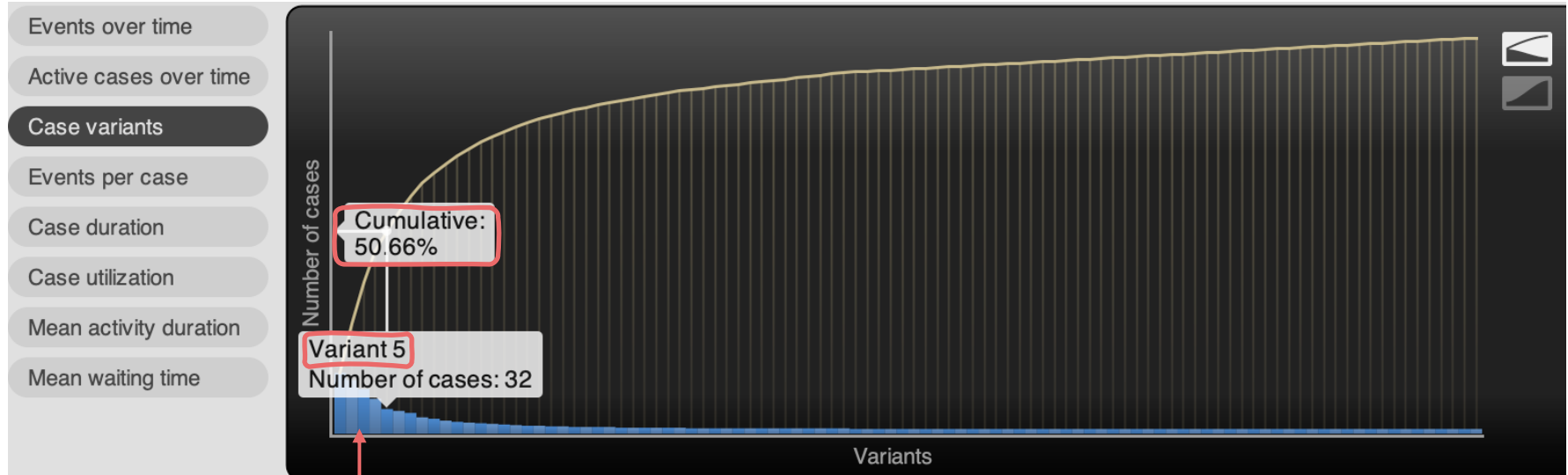
# Inspecting a Variant (i.e., the sequential behaviour)

The sequence of activities captures some important aspects of a case, e.g.:

- **missing** steps (e.g., *pull request was not approved*)
- **unexpected** steps (e.g., *pull request build failure*)
- unexpected **repetitions** of steps (e.g., *pull request was approved twice*)
- unexpected **order** of steps (e.g., *GitHub issue was created after the corresponding pull request was approved*)



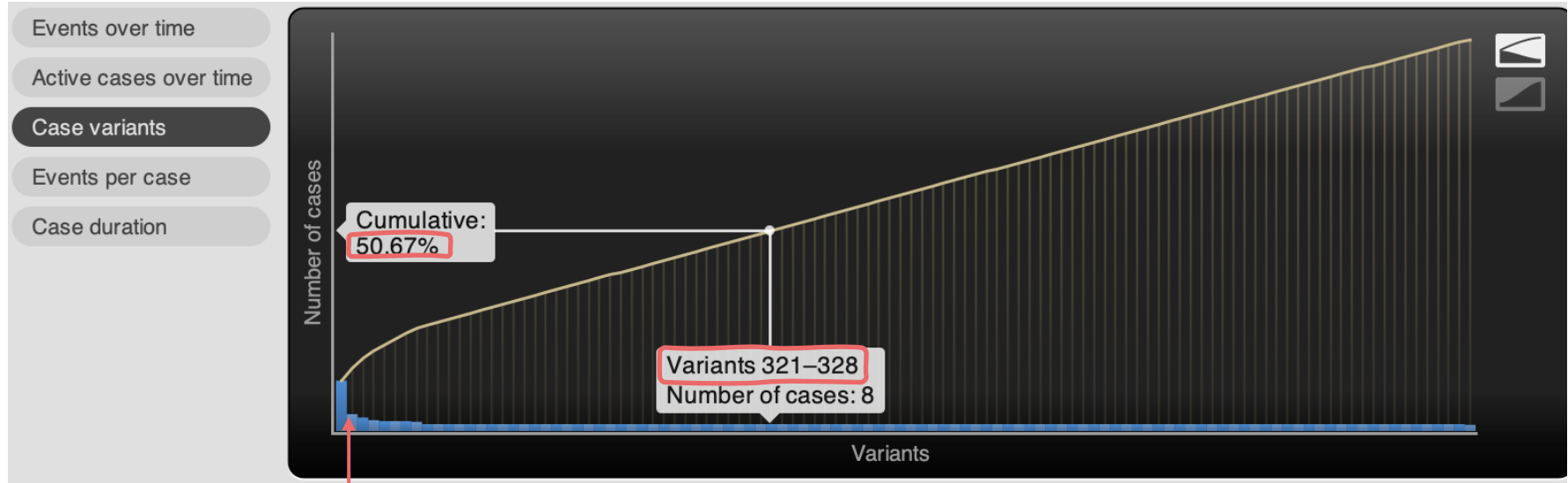
# Variant Distribution (PurchasingExample – 98 variants)



*top variants  
(variants with the highest number  
of cases)*

with inspection of 5 variants, you have  
seen 50% of the sequential behaviour

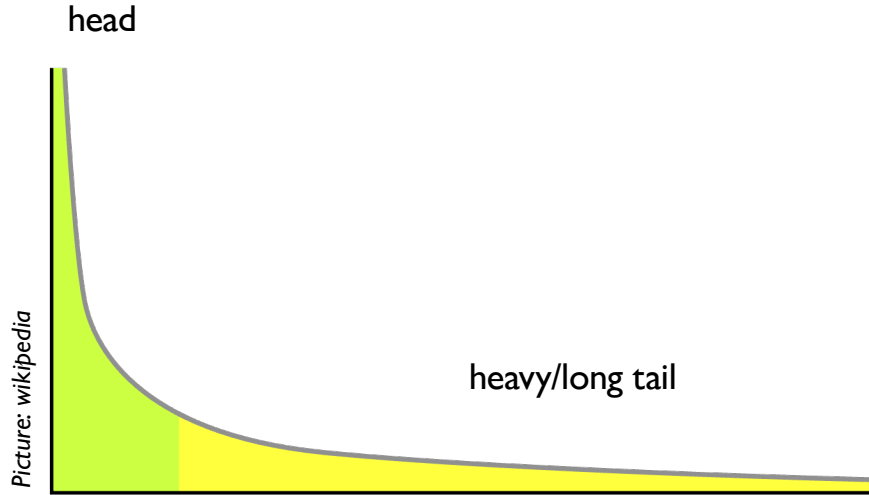
# Variant Distribution (Sepsis – 846 variants)



*top variants  
(variants with the highest number  
of cases)*

to see 50% of the sequential behaviour,  
you need to inspect more than 320  
variants

# Aside: Power Laws



*a very small number  
of items can carry  
substantial mass*

*substantial mass can be  
distributed over a very large  
number of items*

## Other examples: Popularity of

- web store order items
- service requests in a call center
- features in a software product
- social media postings

This affects, e.g., an automation strategy: If automating each service requests incurs approx. the same cost, how many service requests should be automated?

## Further popular science reading:

- C. Anderson: [The Long Tail: Why the Future of Business is Selling Less of More](#) (2006)
- N. Taleb: [The Black Swan](#) (2007)
- G. West: [Scale](#) (2017)

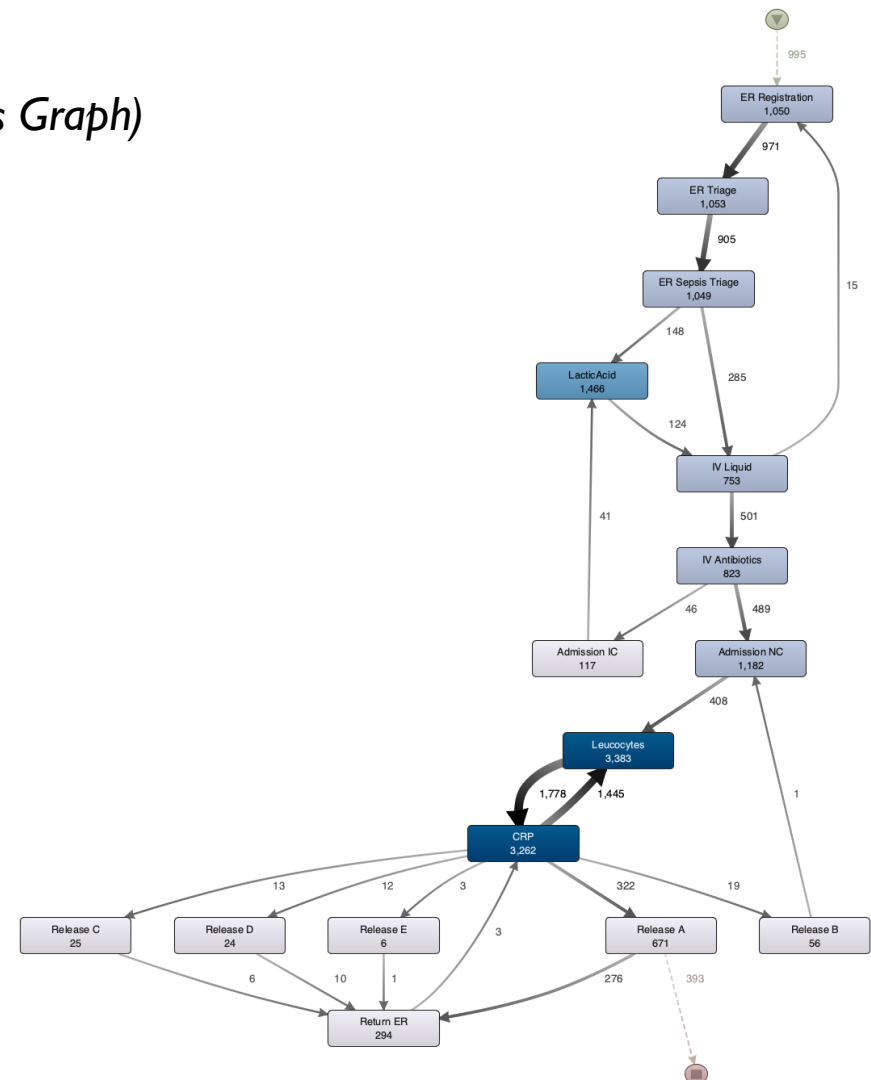
# Process Maps

*(a.k.a. DFGs - Directly-Follows-Graphs)*

# Recall: Process Map (a.k.a. Directly-Follows Graph)

Natural entry point for a new data set

- shows process activities
- their ordering in the log
- their frequencies

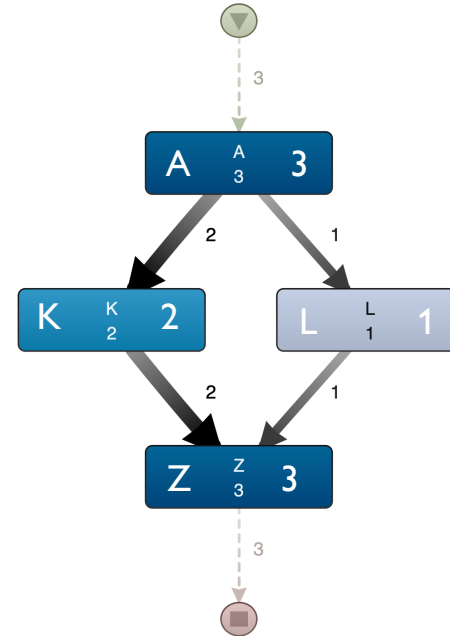


# Directly-Follows-Graph (DFG)

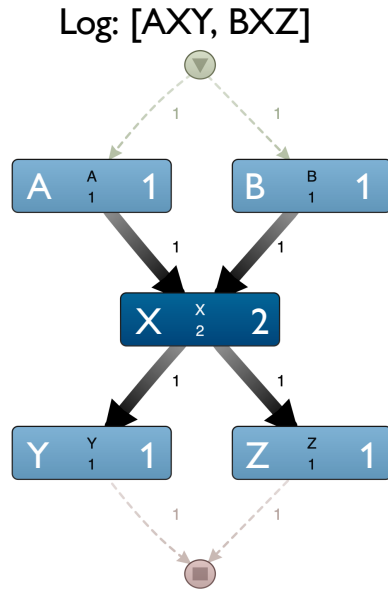
If  $a_1, a_2, a_3, \dots, a_n$  is a sequential variant, then each pair  $(a_i, a_{i+1})$  is called a *log transition*

The DFG shows the set of all log transitions together with their frequency in the log.

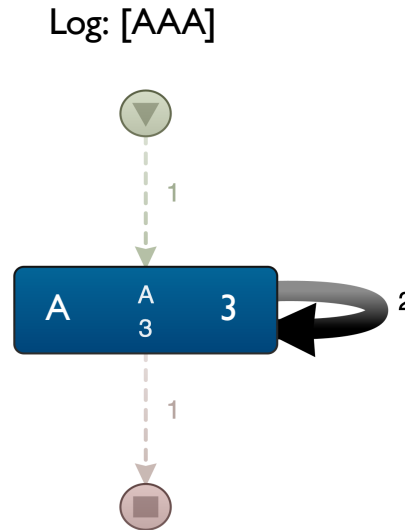
DFG for the log [AKZ, AKZ, ALZ]



# DFG has Limited Variant (i.e., path) Information

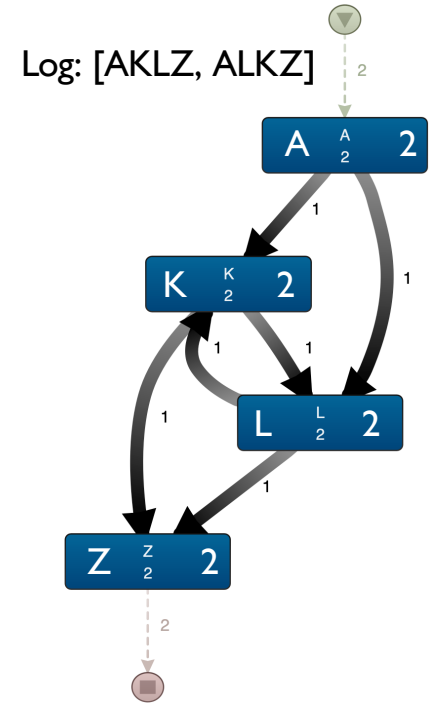


*AXZ, BXY in DFG but not in log*



*Loop in a case, frequencies do not apply to number of cases*

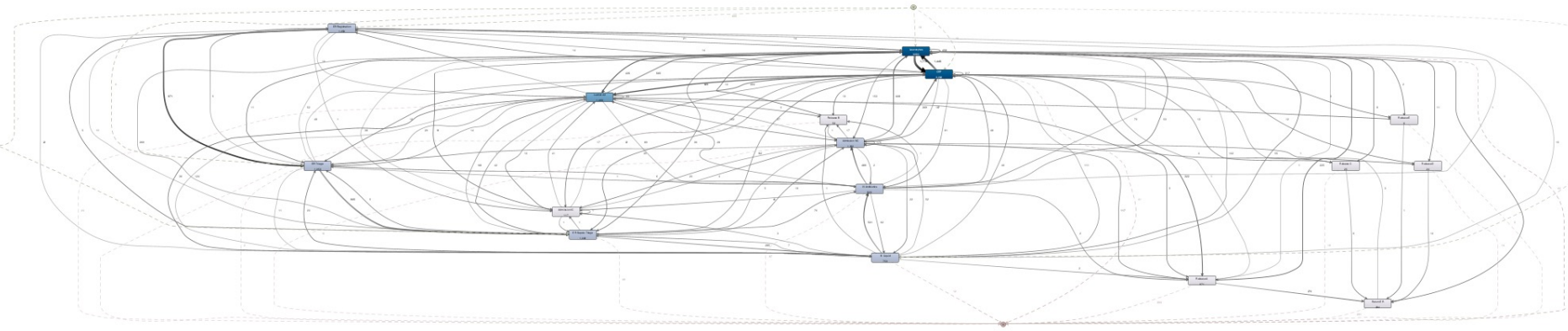
*In Disco, you can also show case frequency*



*Loop in DFG, but not in any case*

*In Disco, you can also show max. repetitions*

# DFG for Sepsis log



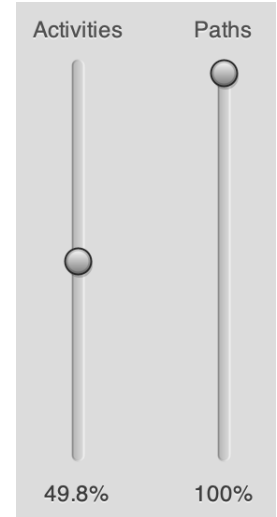
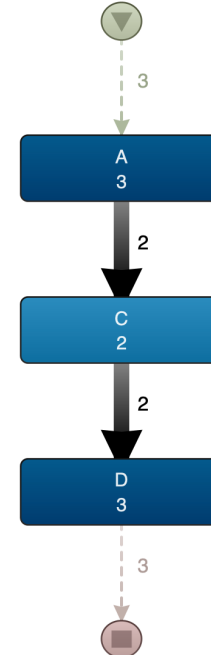
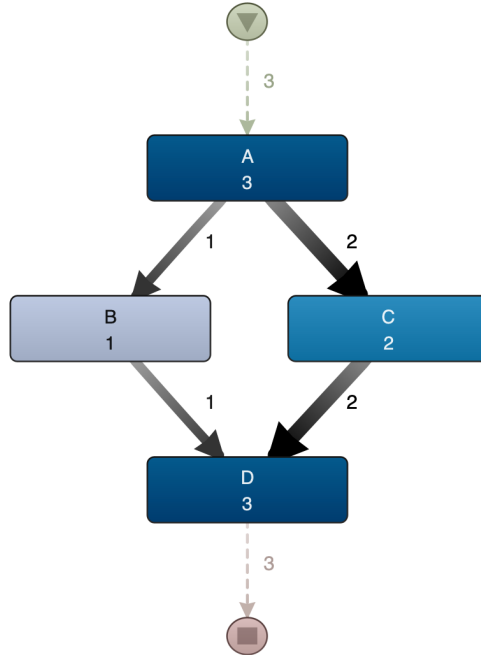
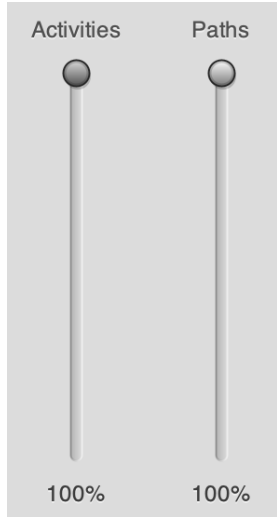
The *unfiltered* DFG of a typical real event log always looks like that. To be human consumable, a DFG needs filtering.

# DFG Filtering - Theory

- **By Activity:** Only show activities in the DFG that occur in the log more than a given threshold
- **By Transitions:** Only show transitions in the DFG that occur in the log more than a given threshold
- **By Variants:** Filter the log to contain only cases of a given set of variants before creating the DFG

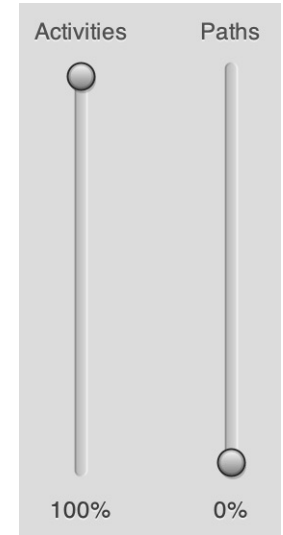
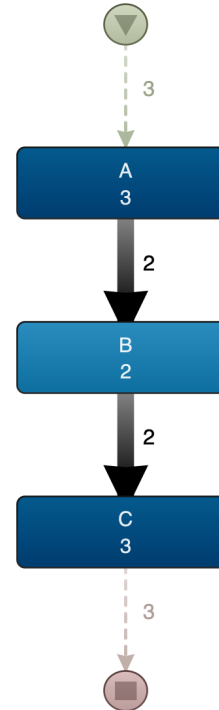
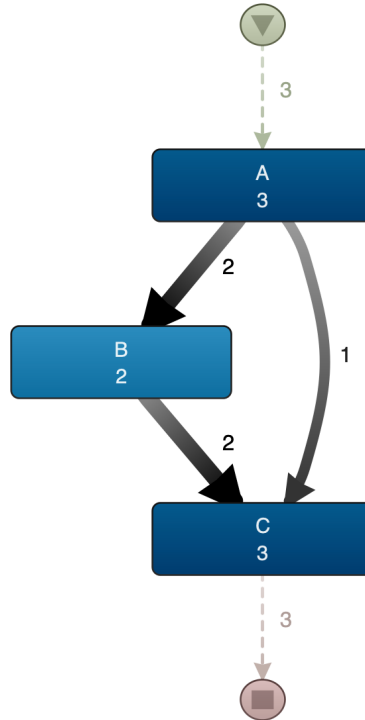
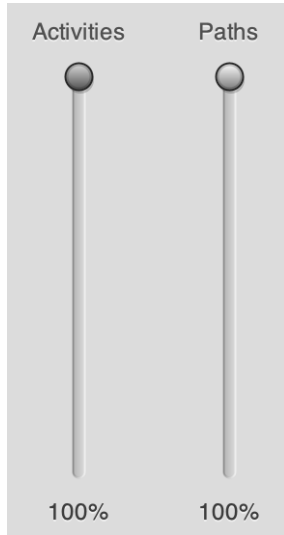
# DFG Filtering – Practice: Activity Filter

What is wrong here?

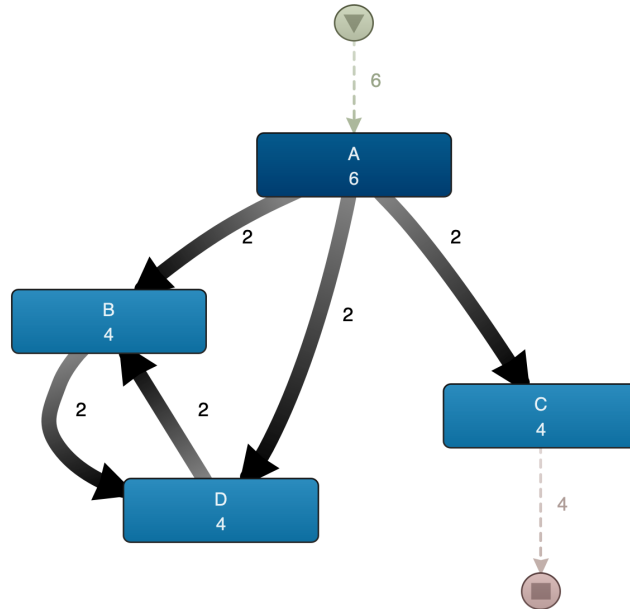


# DFG Filtering – Practice: Path Filter

*What is wrong here?*



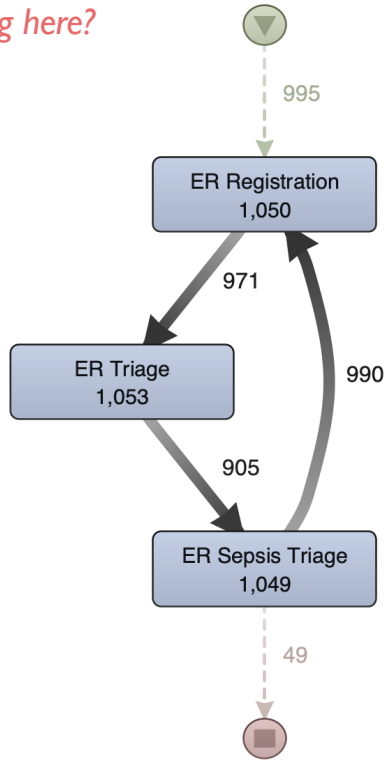
# DFG Filtering – Practice: Activity and Path Filter together



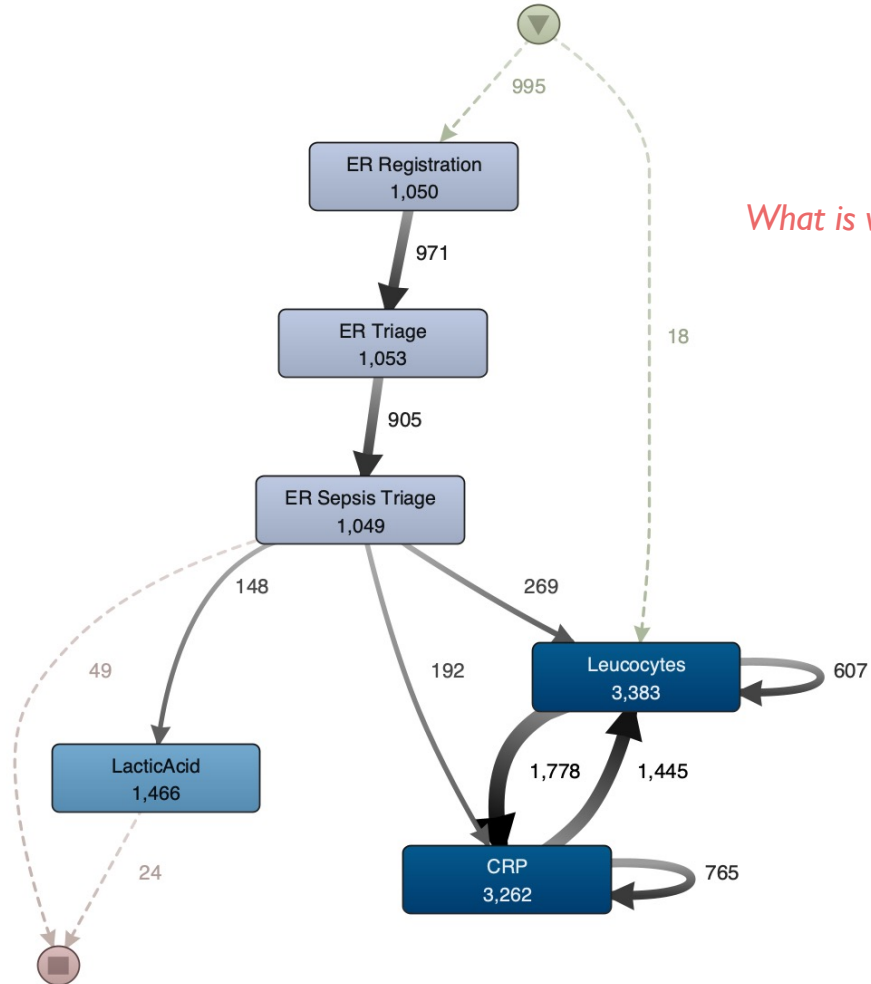
*What is wrong here?*

# Examples from real logs

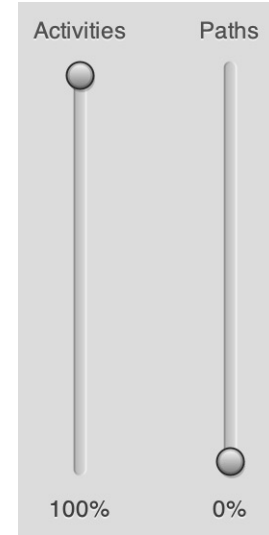
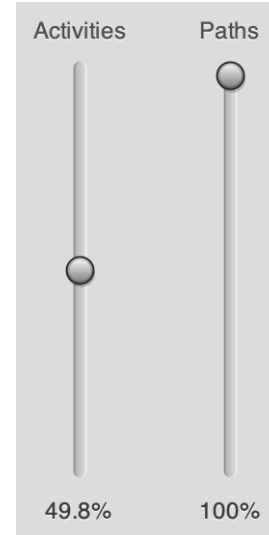
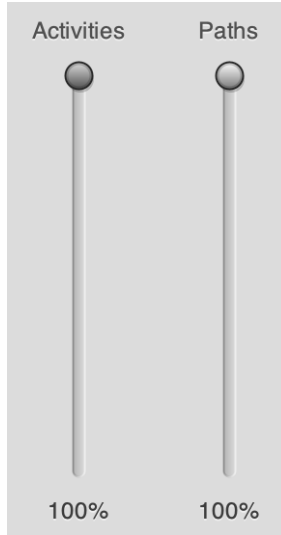
What is wrong here?



What is wrong here?



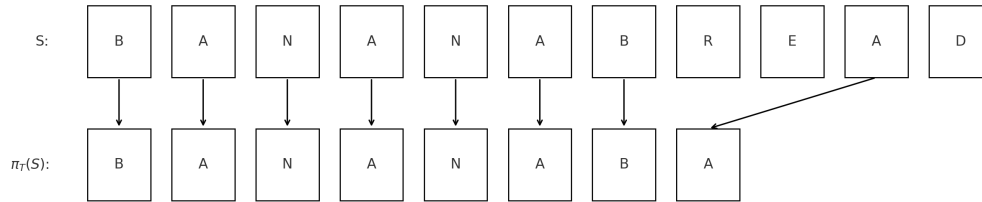
# DFG Filtering – Recommendation



# Sequence Projection

(filtering with a set of events *in all cases*)

Projection of a sequence S onto the subsequence of subset T  
Example: S = BANANABREAD, T = {A, B, N}

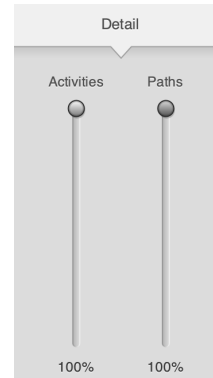


Select the activities of a particular **object** (or subprocess)

- e.g. the **main business object** of the process

Apply filter before creating DFG

- Use DFG with both sliders at 100%*



Filter settings for: SepsisCases2020EventLog

Active filters

Attribute  
Removes events by attribute

click to add filter...

Filter by: Activity

Filtering mode:

- ☒ Keep selected
- ☐ Mandatory
- ☐ Forbidden

This filter removes all events that do not have one of the selected values.

Event values: (10 of 16 selected)

- ☒ Admission IC
- ☒ Admission NC
- ☒ CRP
- ☒ ER Registration
- ☒ ER Sepsis Triage
- ☒ ER Triage
- ☒ IV Antibiotics
- ☒ IV Liquid
- ☒ LacticAcid
- ☒ Leucocytes
- ☒ Release A
- ☒ Release B
- ☒ Release C
- ☒ Release D
- ☒ Release E
- ☒ Return ER

# Exercise (5 mins)

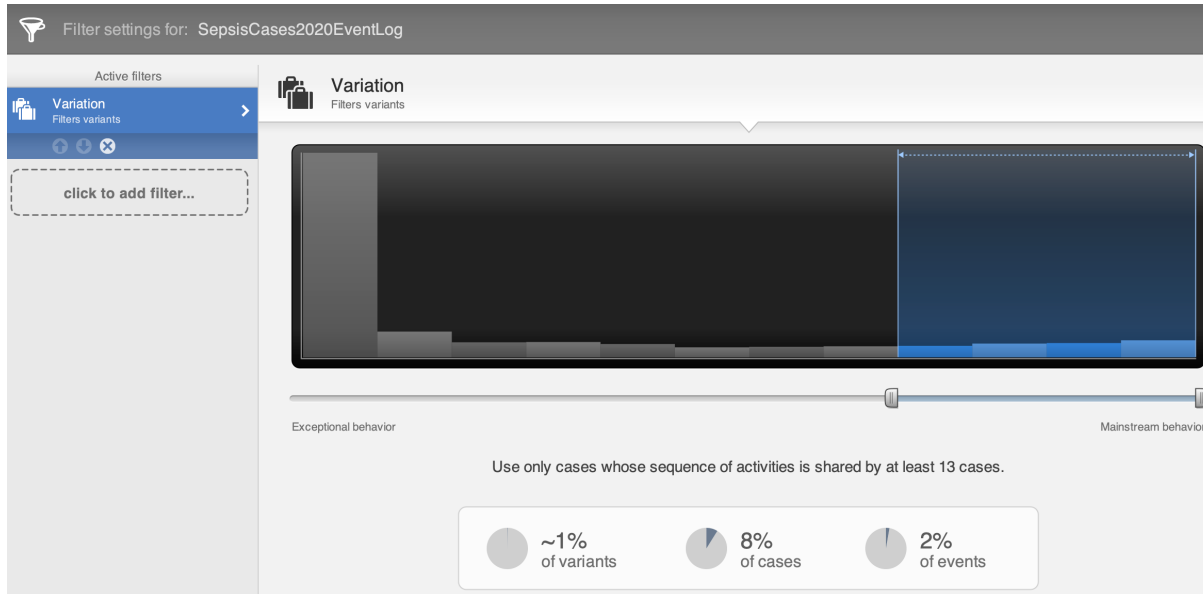
In Disco, apply a filter that selects only the following administrative activities:

- ER Registration, ER Sepsis Triage, ER Triage
- Admission IC, Admission NC
- Release A, Release B, Release C, Release D, Release E

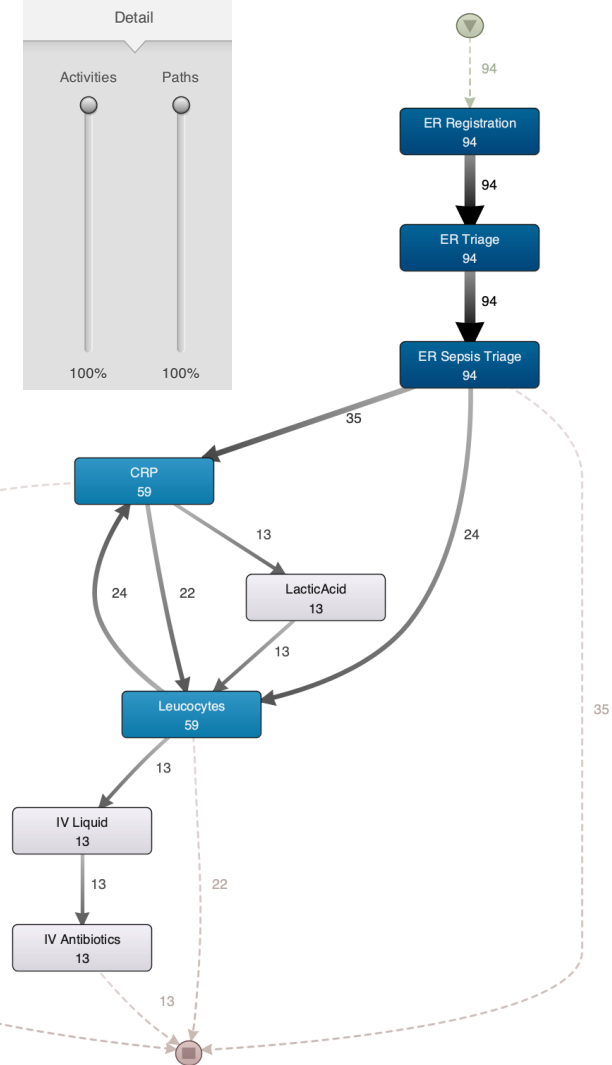
Inspect the process map with sliders at 100%. Identify transitions (i.e., arrows) with metric *absolute frequency* or *case frequency* that are unexpected to you and should be investigated further.

Use the *Follower filter* to determine how many cases exhibit that unexpected behavior.

# Variant Filtering



At 100% detail, the DFG can keep its clear (but limited) semantics. Complexity can be reduced by filtering variants (e.g. keep only top variants).



# Variant Filtering (Flexible Variant Selection)

Filter settings for: SepsisCases2020EventLog

Active filters

- Attribute  
Removes events by attribute

click to add filter...

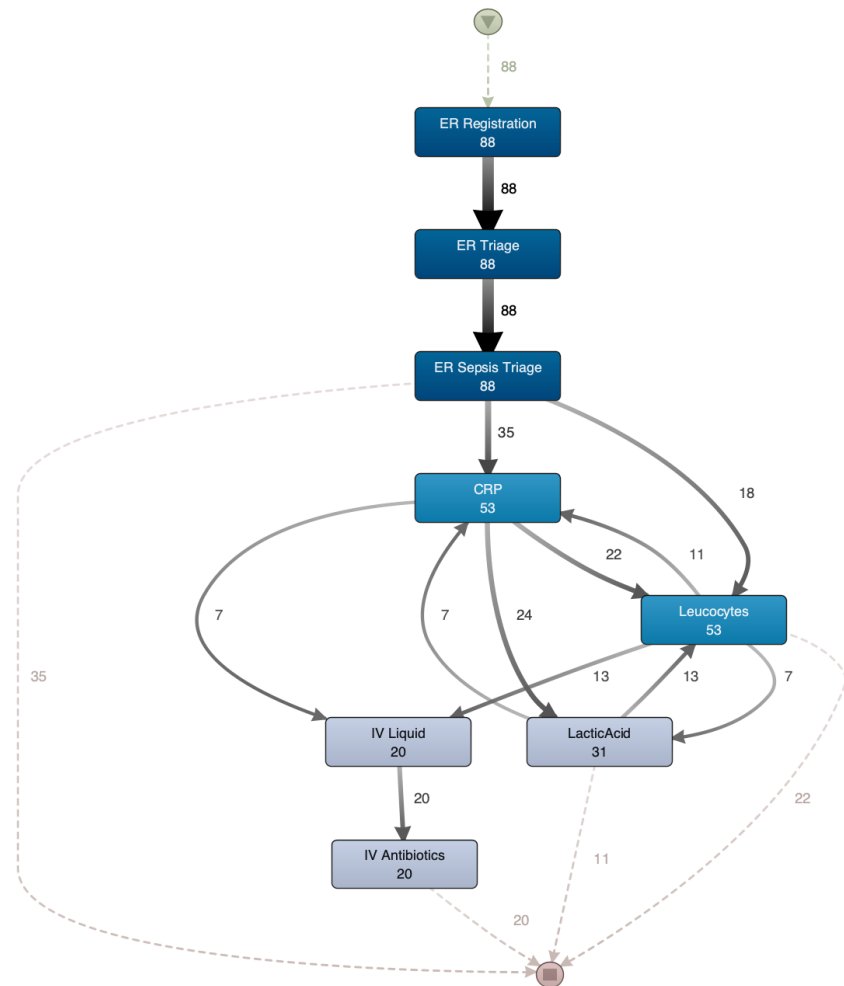
Filter by: Variant

This filter keeps only cases from the selected variants.

Event values: (5 of 846 selected)

- ☒ Variant 1
- ☐ Variant 2
- ☒ Variant 3
- ☒ Variant 4
- ☒ Variant 5
- ☐ Variant 6
- ☒ Variant 7
- ☐ Variant 8

*select individual variants by number*



# Case Filter

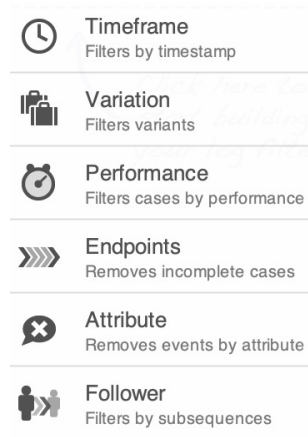
*To maximally preserve path information, apply only filters that preserve entire paths, i.e., full cases*

Apply filter before creating DFG

- *Use DFG with both sliders at 100%*

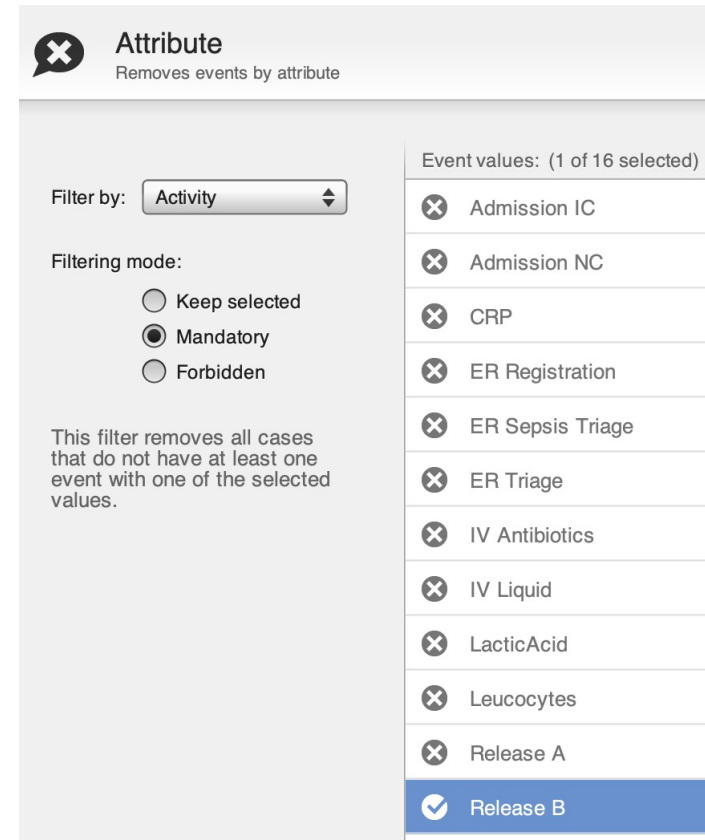
Various case filters are available in *Disco*, e.g.

- Selected sequential variants
- Mainstream/ exceptional variants (*next slide*)
- ...



The screenshot shows a vertical list of filter options in the Disco interface:

- Timeframe**: Filters by timestamp (clock icon)
- Variation**: Filters variants (briefcase icon)
- Performance**: Filters cases by performance (stopwatch icon)
- Endpoints**: Removes incomplete cases (three arrows icon)
- Attribute**: Removes events by attribute (speech bubble with X icon)
- Follower**: Filters by subsequences (two people icon)



The screenshot shows the 'Attribute' filter configuration panel in Disco. It includes a 'Filter by' dropdown set to 'Activity', a 'Filtering mode' section with radio buttons for 'Keep selected', 'Mandatory' (selected), and 'Forbidden', and a list of event values. A descriptive text states: 'This filter removes all cases that do not have at least one event with one of the selected values.'

Event values: (1 of 16 selected)

|                                     |                  |
|-------------------------------------|------------------|
| <input type="checkbox"/>            | Admission IC     |
| <input type="checkbox"/>            | Admission NC     |
| <input type="checkbox"/>            | CRP              |
| <input type="checkbox"/>            | ER Registration  |
| <input type="checkbox"/>            | ER Sepsis Triage |
| <input type="checkbox"/>            | ER Triage        |
| <input type="checkbox"/>            | IV Antibiotics   |
| <input type="checkbox"/>            | IV Liquid        |
| <input type="checkbox"/>            | LacticAcid       |
| <input type="checkbox"/>            | Leucocytes       |
| <input type="checkbox"/>            | Release A        |
| <input checked="" type="checkbox"/> | Release B        |

*Some filter may be intuitive to interpret, but fail to reduce DFG complexity.*

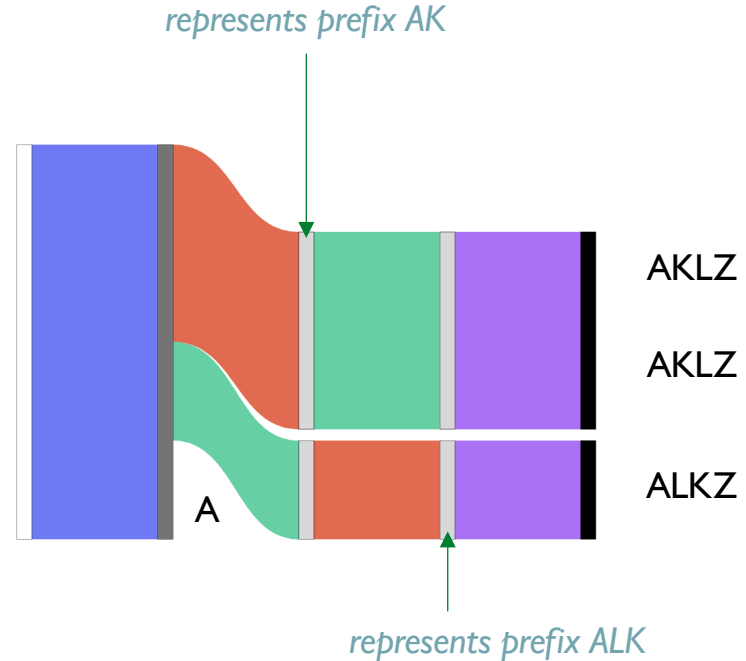
# Better Process Maps

*Linear Continuous Maps (LCM) – Visualization through Sankey Diagrams*

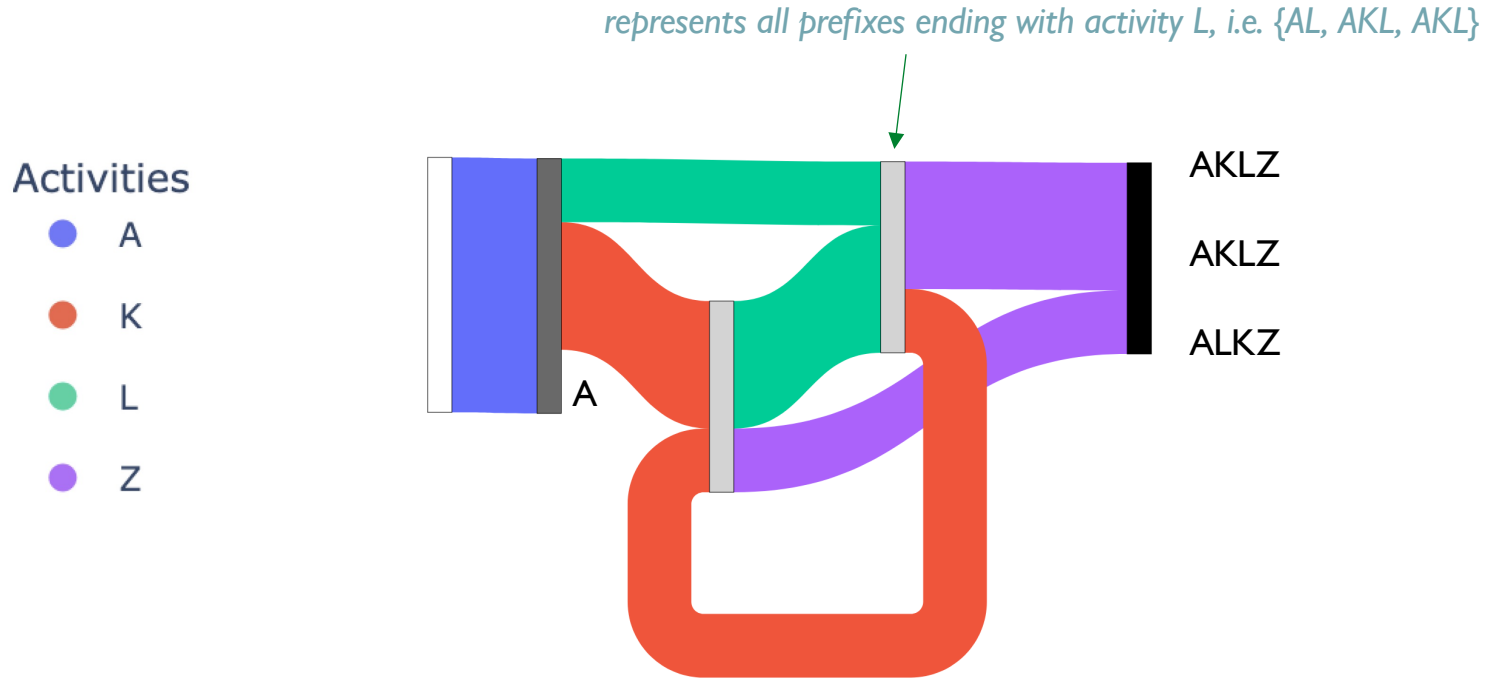
# Prefix Tree (Sequence Based) Event log = {A, AKLZ, AKLZ, ALKZ}

## Activities

- A
- K
- L
- Z



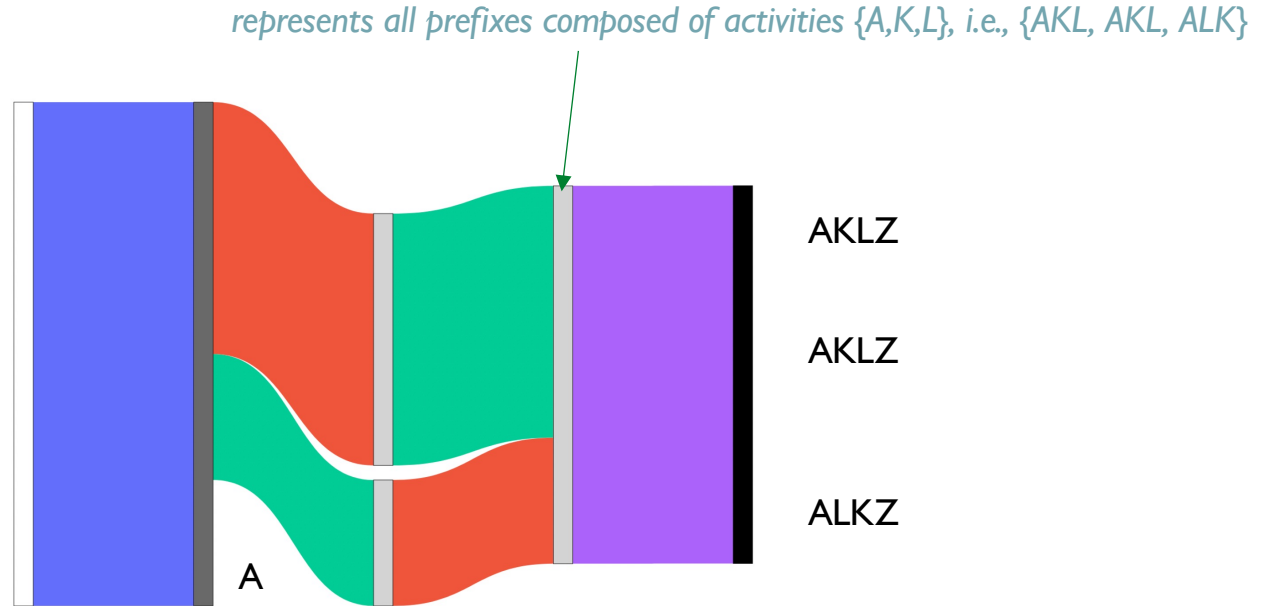
DFG (Last Activity Abstraction) Event log = {A, AKLZ, AKLZ, ALKZ}



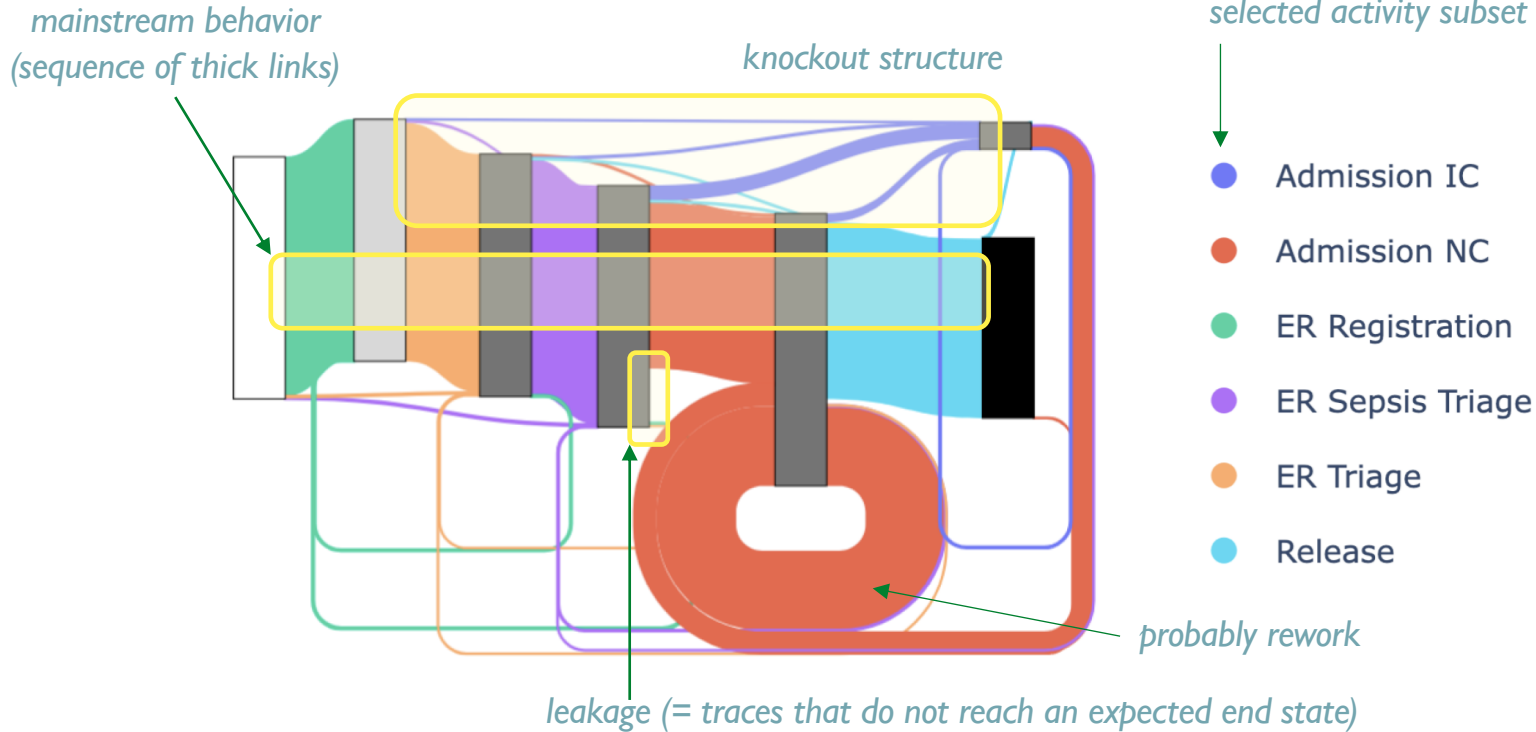
# Set Abstraction Event log = {A, AKLZ, AKLZ, ALKZ}

## Activities

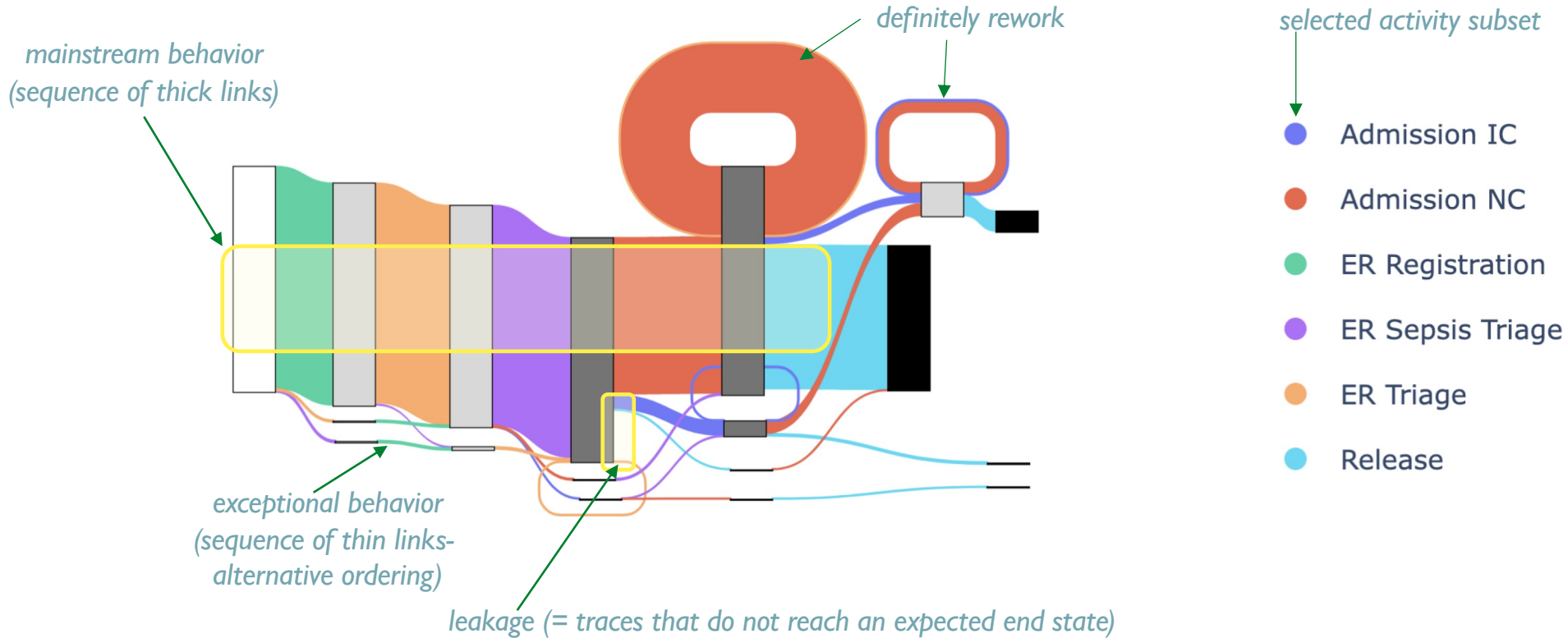
- A
- K
- L
- Z



# LCM for Sepsis (*admin subprocess, DFG version*)



# LCM for Sepsis (*admin subprocess, Set abstraction*)



# Usage

- Download from [GitHub](#)
- Use as `marimo edit marimo_app.py` (uv not needed)
- Change the event log in first code cell

# Process Modeling

*Specify behaviour of a process as a **descriptive**, **predictive**, or **prescriptive** model.*

# BPMN (= Business Process Model and Notation)

- Dominant standard for modeling business processes
  - Alternative: *UML Activity Diagrams*
- Widely adopted in practice for
  - modeling and
  - *execution* (runs on dedicated engines: *Camunda*, *IBM*, *SAP*, *Oracle*, ...)

# Petri Nets

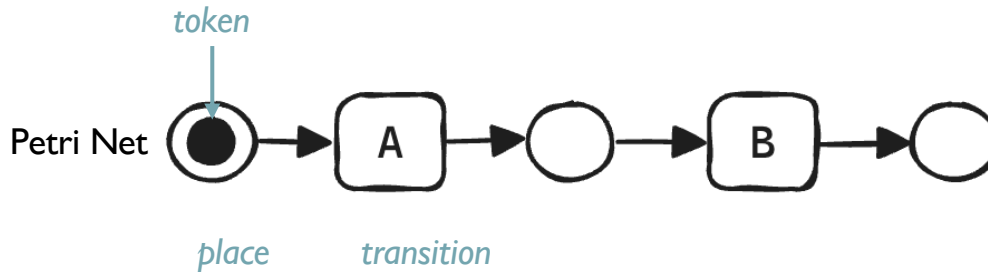
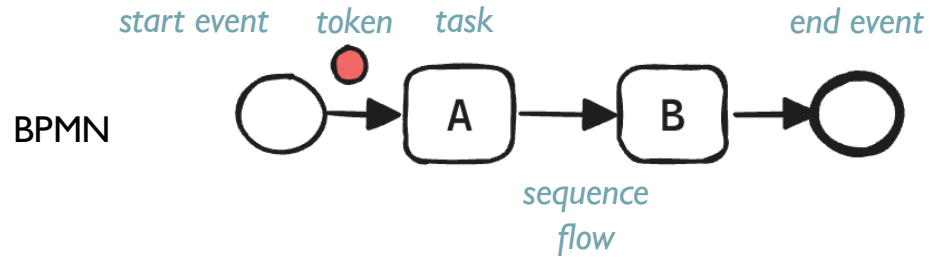
- Modeling formalism for concurrent processes
- Widely known in academia
- Many analysis algorithms known
- Useful ...
  - to understand concurrency foundations
  - as an interchange between high-level modeling languages, analysis algorithms, and other tools

*Inventor of Petri Nets*



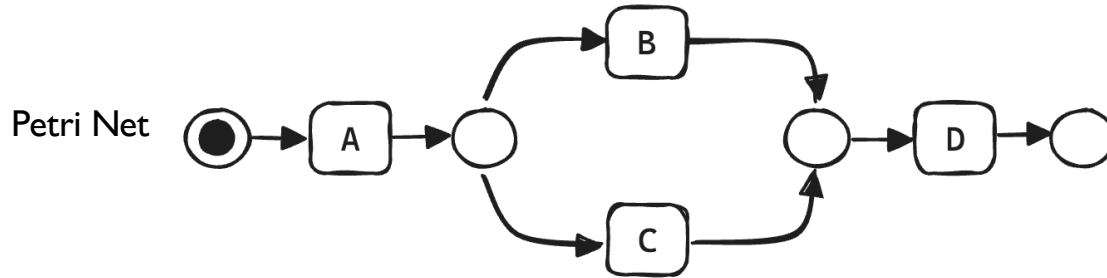
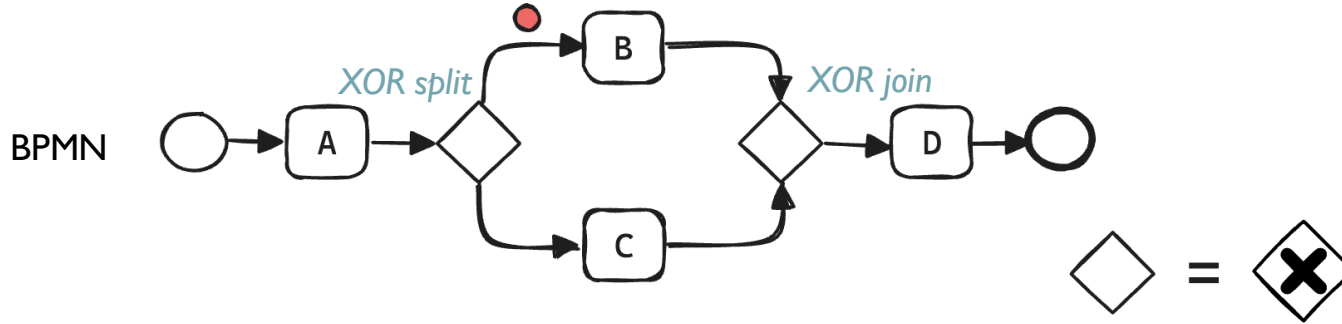
*Carl Adam Petri*

# Sequence



Set of *traces*: {AB}

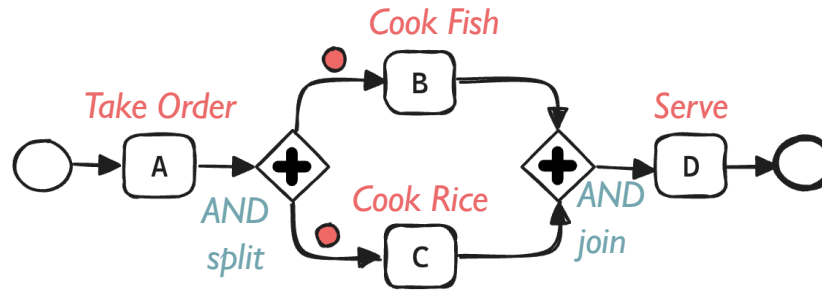
# Free Choice (XOR)



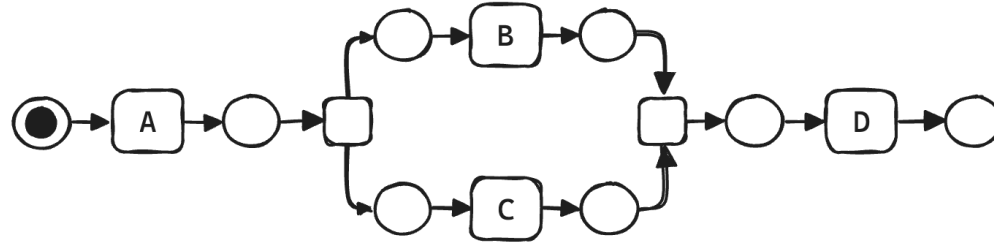
Set of traces: {ABD, ACD}

# Synchronization (AND)

BPMN

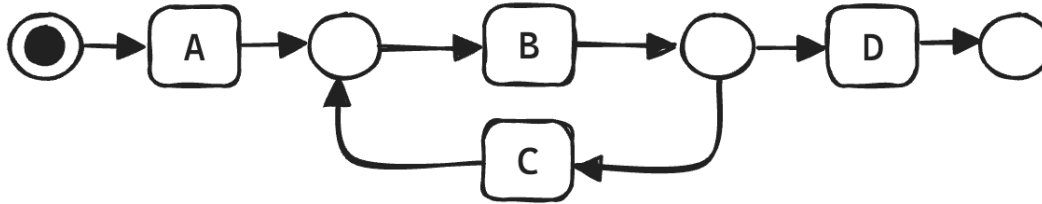
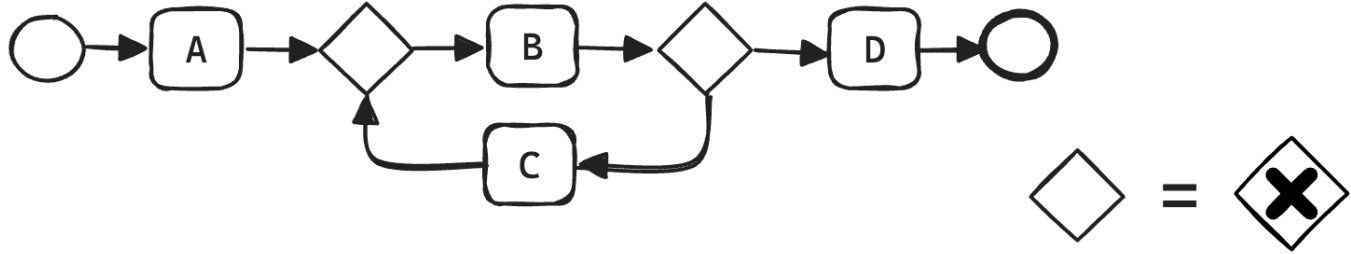


Petri Net



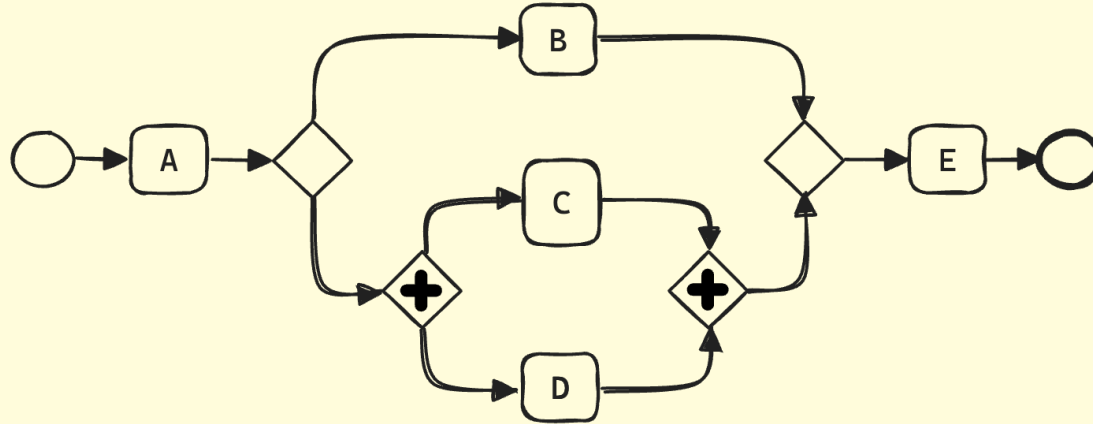
Set of traces: {ABCD, ACBD}

# Loop



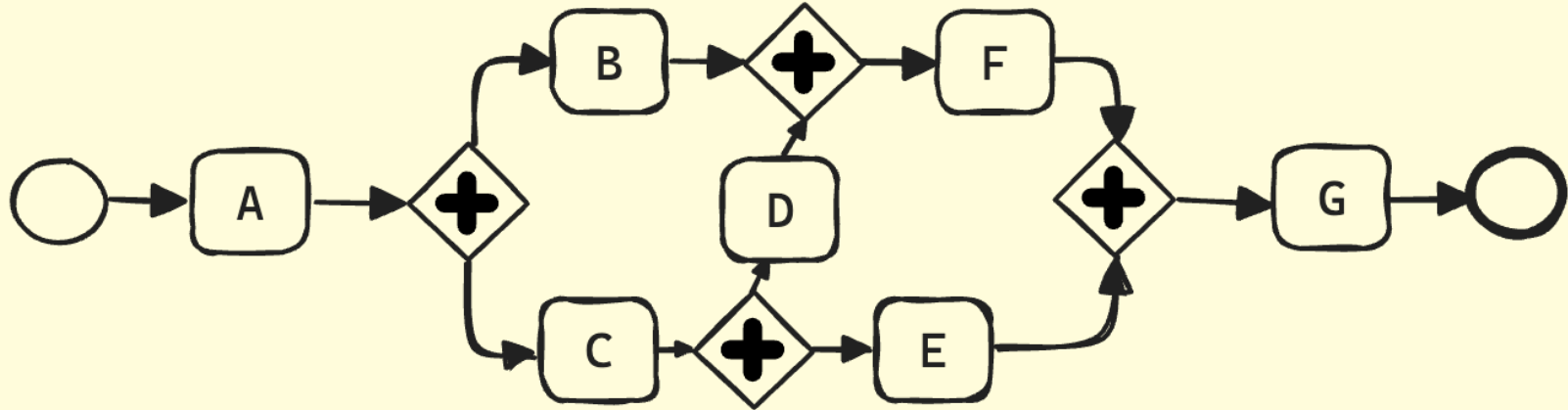
Set of traces: {ABD, ABCBD, ABCBCBD...}

# Composition I



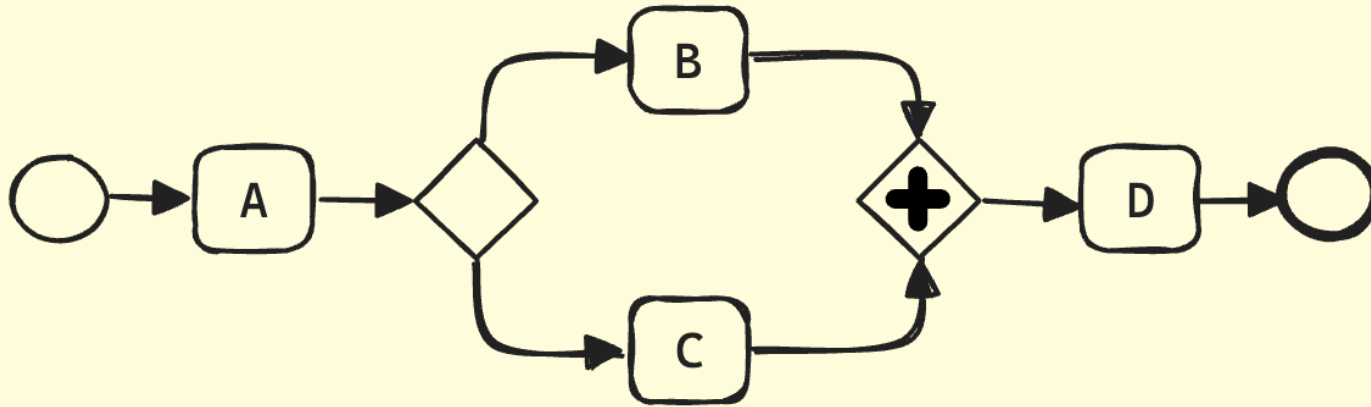
Set of traces: ?

# Composition II

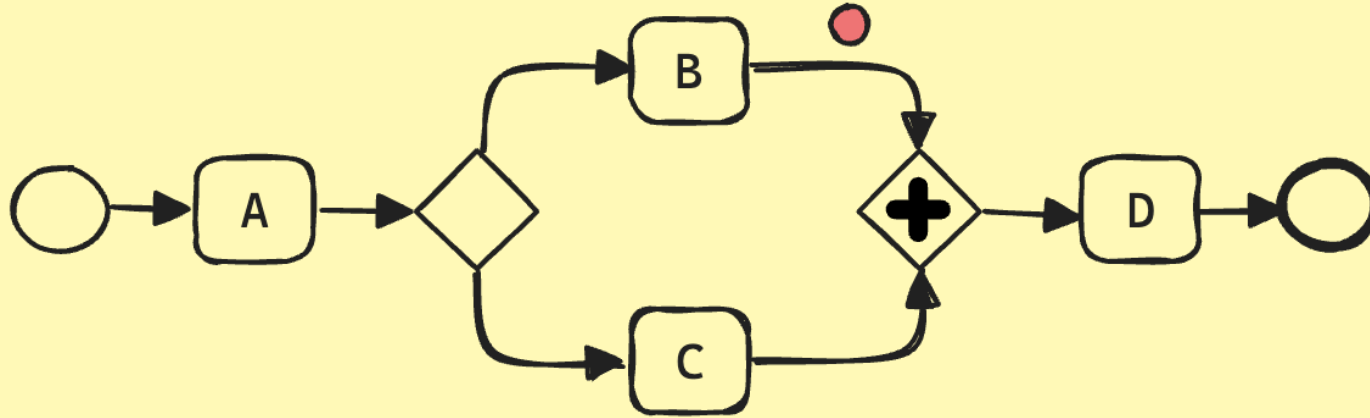


Is ACDBFEG a trace? ACBEFDG?

# What is wrong here?

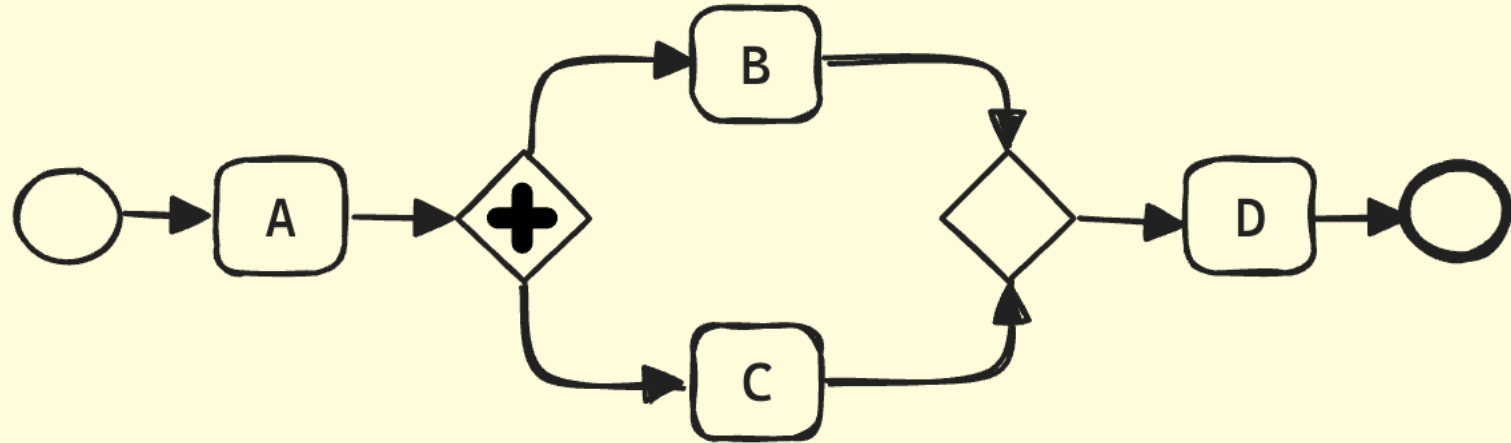


# A Deadlock

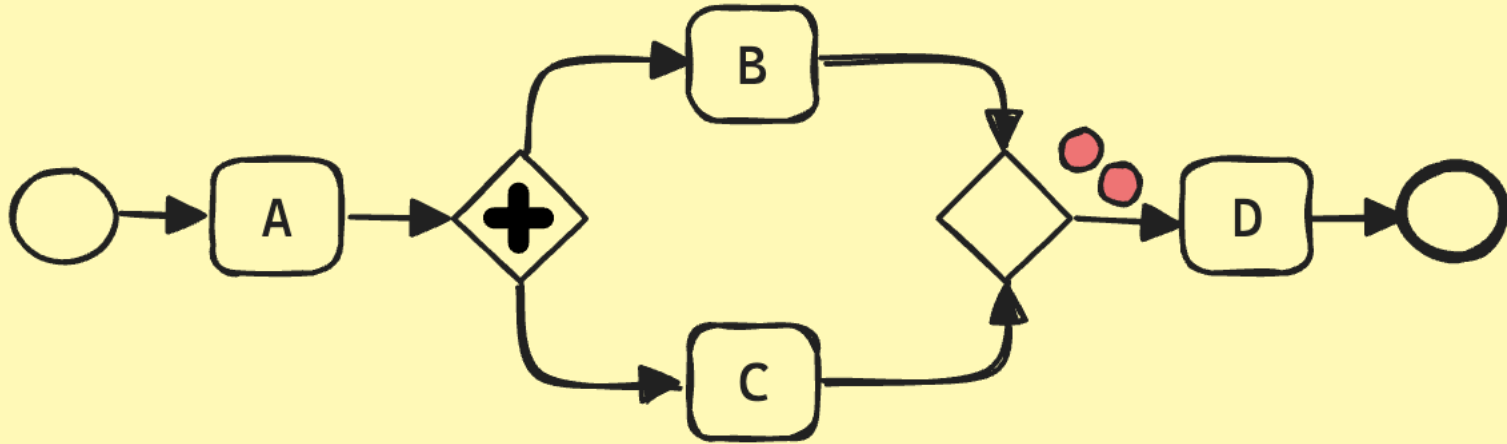


Deadlock = an AND-join has an incoming token as well as an incoming flow that *cannot* get a token.

# What is wrong here?

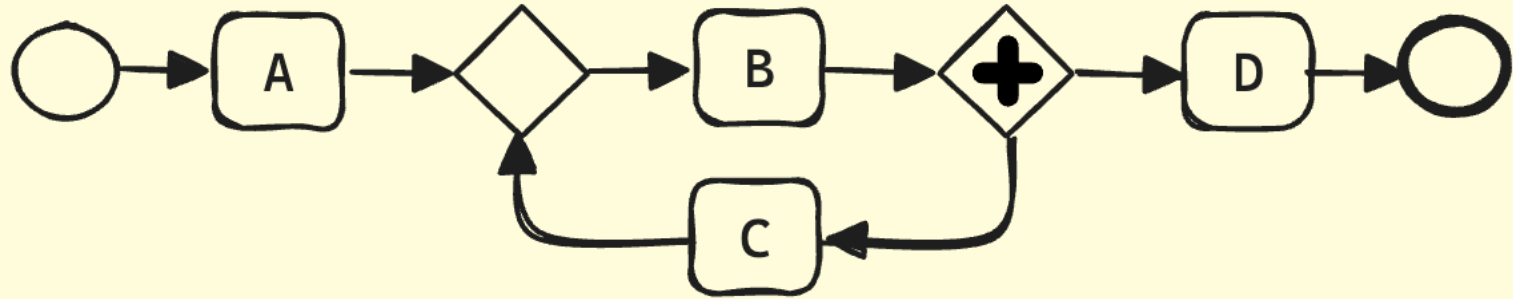


# Lack of Synchronization

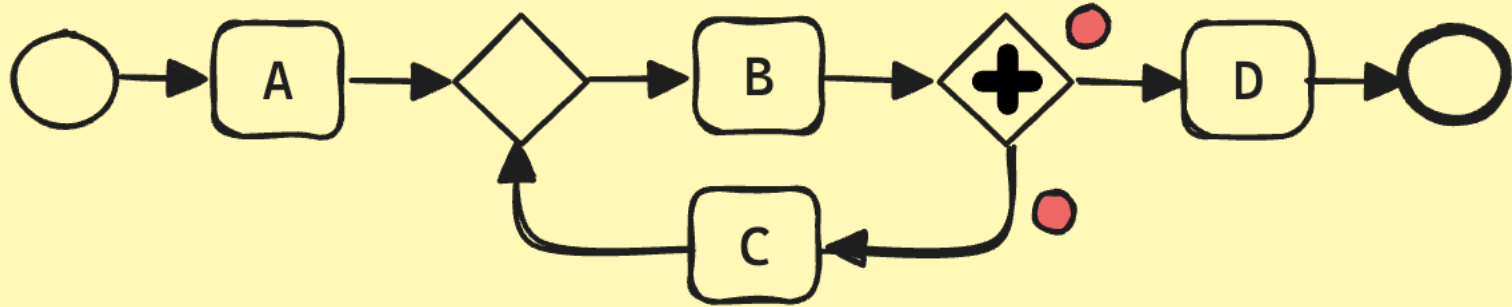


Lack of Synchronization = a sequence flow has more than one token,  
(therefore each subsequent task gets executed concurrently to itself)

# What is wrong here?



# Unbounded Lack of Synchronization



Lack of Synchronization = a sequence flow has more than one token,  
(therefore each subsequent task gets executed concurrently to itself)

# Soundness

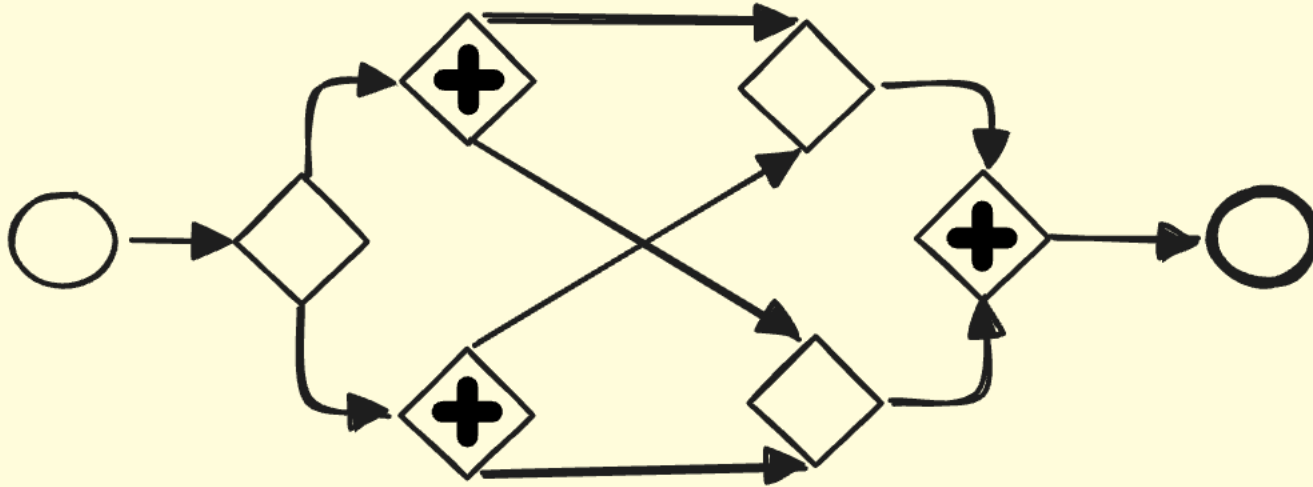
A *workflow graph* is a BPMN graph that

- has a unique start event and a unique end event
- all nodes lie on a path from the start to the end event
- each gateway is either an XOR or an AND gateway.

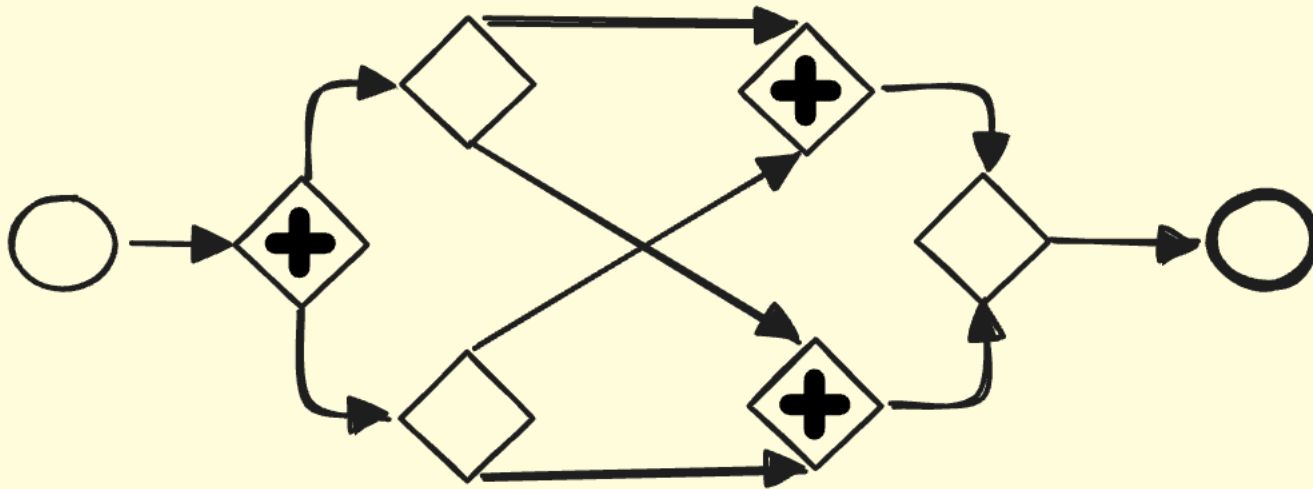
A workflow graph is called *sound* if it does not have any deadlock and lack of synchronization.

**Theorem:** Soundness of a workflow graph can be decided in cubic time.

# Is this workflow graph sound?

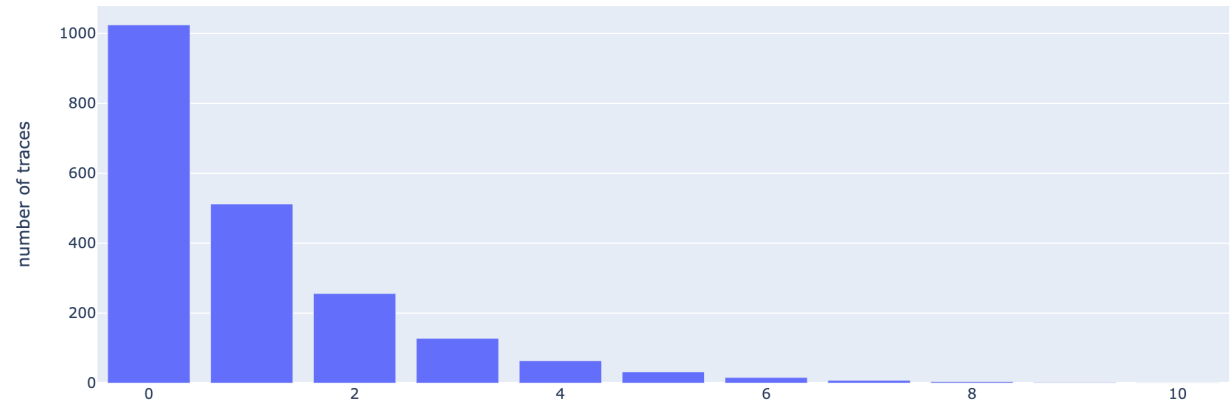
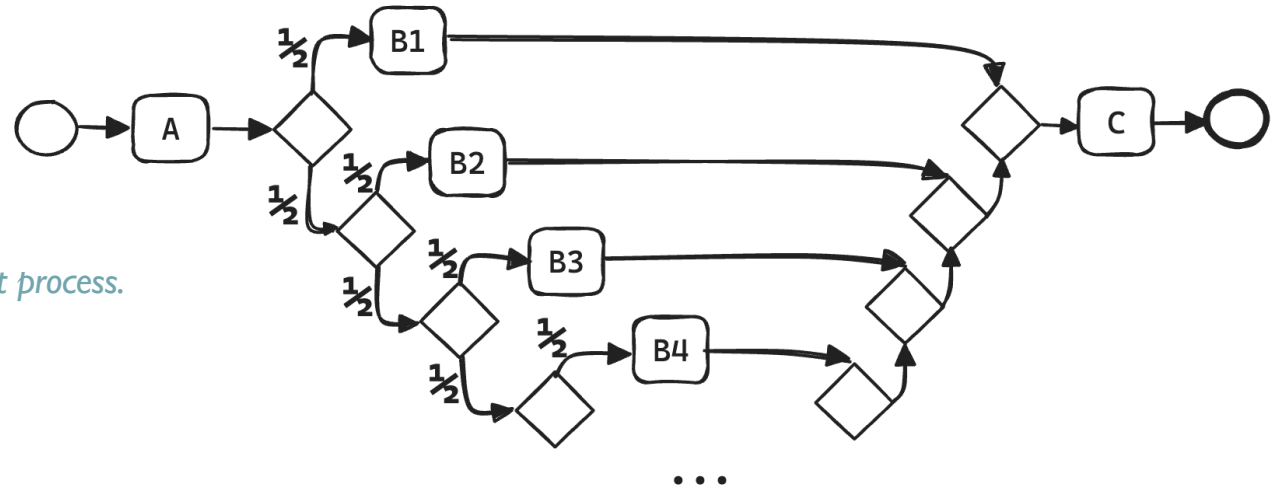


# Is this workflow graph sound?

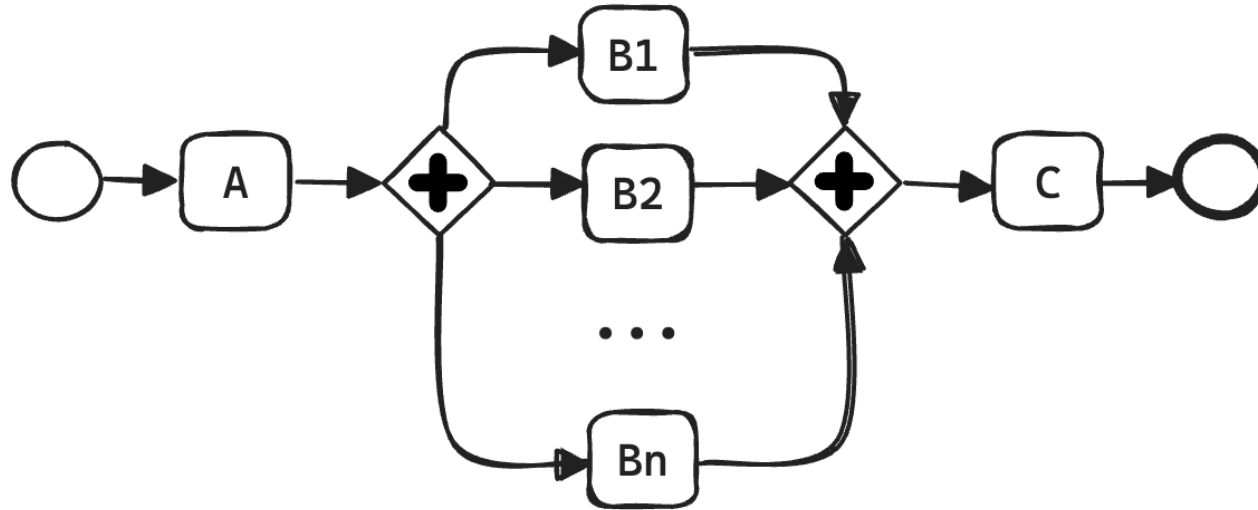


# Combinatorics

*A knock-out process.*



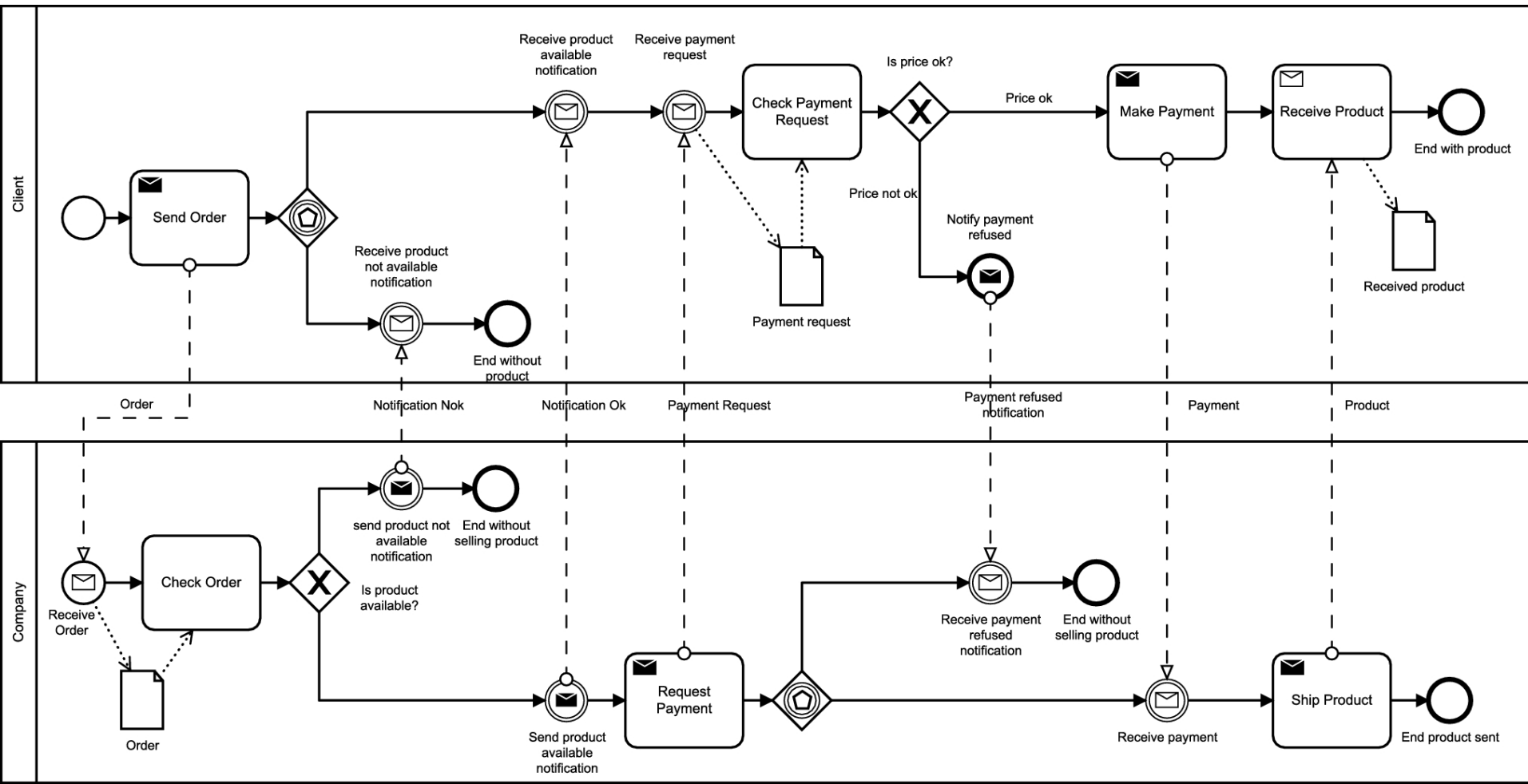
# Concurrency



$n$  concurrent tasks create  $n!$  different traces (sequential variants) with the exact same set of tasks, which of those variants occur in a log depends on the timing/scheduling in the environment

# Outlook: BPMN Example (*beyond workflow graphs*).

Source: [Compagnucci et al.](#)



# Process (Model) Induction

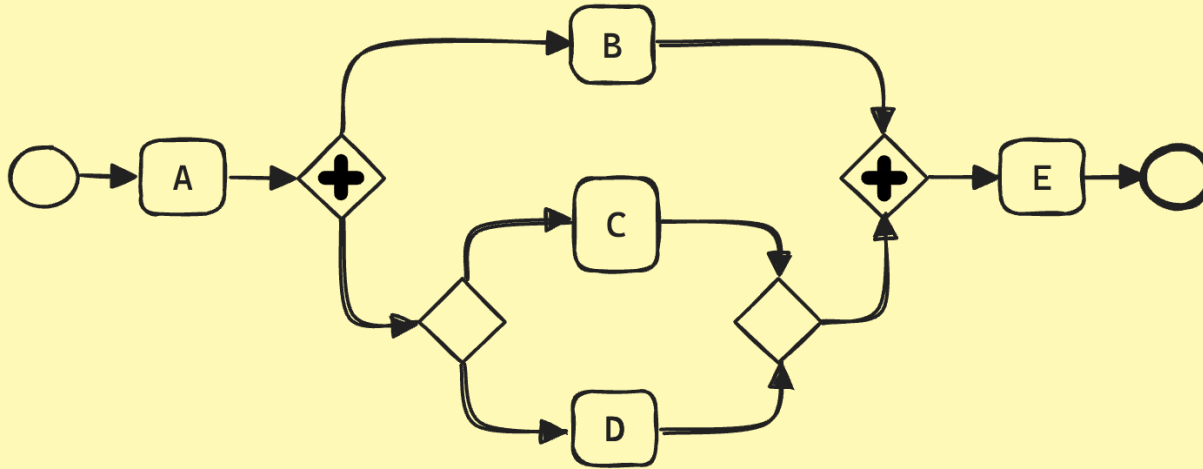
*= Induce a process model from a set of traces, i.e., from an event log*

# Process Induction

How does the process look like that has this set of traces: {ABCE, ACBE, ABDE, ADBE}?

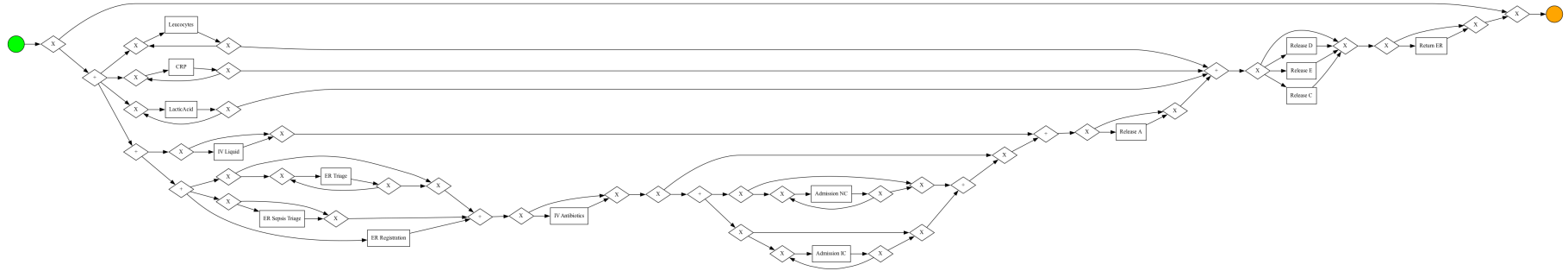
# Process Induction

How does the process look like that has this set of traces: {ABCE, ACBE, ABDE, ADBE}?

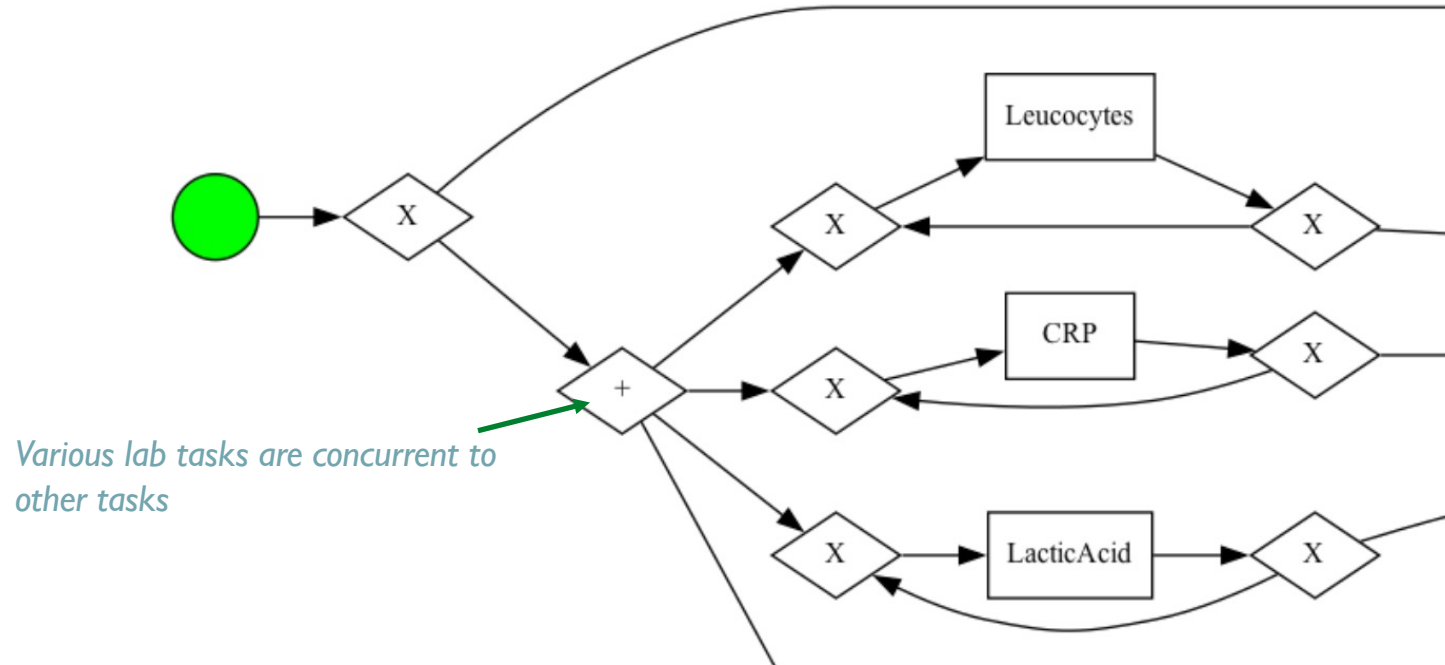


# Inductive Miner (one of the best algorithms, but still quite limited)

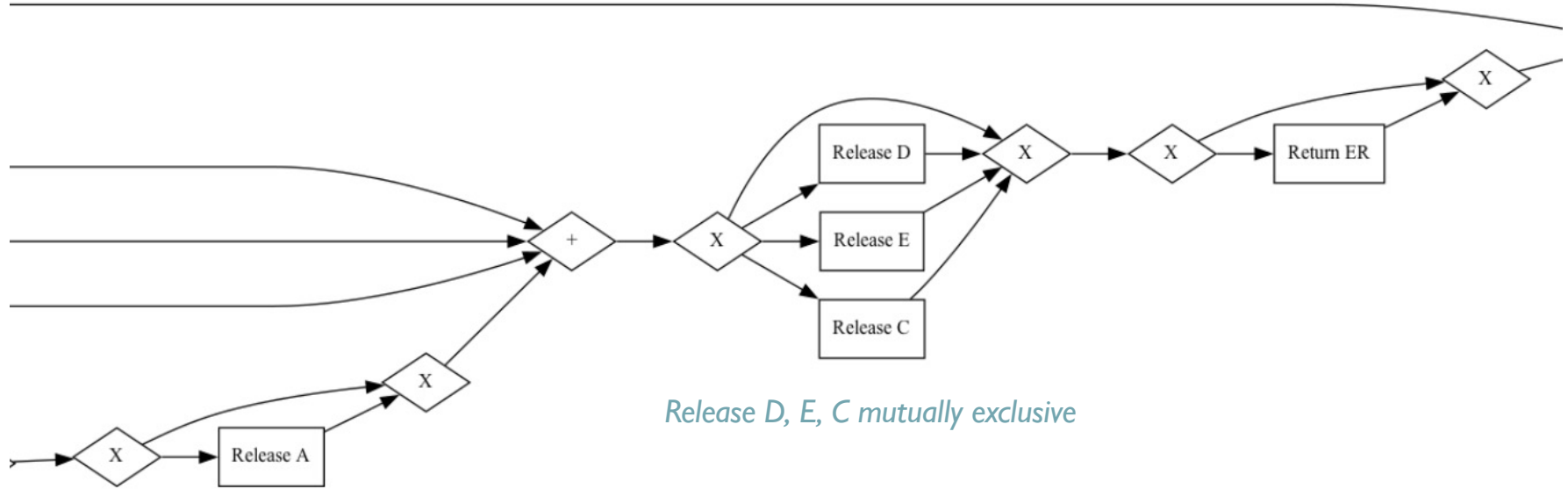
```
bpmn_model = pm4py.discover_bpmn_inductive(event_log, noise_threshold=0.1)
pm4py.view_bpmn(bpmn_model)
```



# Induced Sepsis Process - Detail



# Induced Sepsis Process – Detail 2



*Release D, E, C mutually exclusive*

*Release A appears before other Releases*

*Release B missing*

# Apromore

- You can also use Apromore for process induction. Apromore is a web/cloud application.

## Apromore

- Get an [academic account](#) with your university e-mail
- You can upload and analyze event logs up to 500K rows and up to 25 columns with the academic license
  - Larger event logs will be trimmed automatically on import

# Assignment 1: Tasks from Session 4 Part 1 (Discovery)

1. Inspect the distribution of sequential variants (in *Disco* or *Python*).
  - a) What is the minimum number of sequential variants that is needed to cover 80% of the cases? *[no code required.]*
  - b) Summarize each of the top 4 sequential variants in one sentence. *[no code required.]*
  - c) The **fine object** sub-log is defined by filtering out all events from the event log except the events from the following activities: ['Create Fine', 'Send Fine', 'Insert Fine Notification', 'Add penalty', 'Send for Credit Collection', 'Payment']. Apply this filter to the event log and compare the number of sequential variants after and before filtering. *[no code required.]*
2. Inspect the process map in *Disco* with absolute frequency or the Advanced Process Map (*Linear Continuous Process Mapper*) **for the fine object defined in task 1** above.
  - a) Write down at least three observations that are unexpected to you and hence you would suggest them to be further investigated. Explain briefly why.
  - b) Use a filter to find a sequential variant that exhibits an unexpected behavior. Report the variant as a sequence of activities.
  - c) Inspect a case of the variant reported in b). Can you come up with an explanation for the unexpected behavior?

# Assignment 1: Tasks from Session 4 Part 2 (BPMN)

3. Create a workflow graph model in BPMN that reflects your current (preliminary) understanding of how the ***fine object*** should behave (*You might have seen behavior in the log that does not comply with that understanding. That is ok.*) Use [bpmn.io](https://bpmn.io) to create a *bpmn* file and a *svg* file, start from the template provided in Canvas, which contains all activities that the fine object is involved in. Make sure that your model contains all these activities and make sure your model is sound.